МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА КОМП'ЮТЕРНИХ НАУК

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 122 «Комп'ютерні науки»
(освітня програма «Комп'ютерна графіка та розробка ігор»)
на тему:

«Розробка CRM системи для онлайн школи»

Виконав студент групи КН-22-2
ТАЛЬКО Павло Вікторович

Керівник роботи:
ПАНАРІНА Ірина Володимирівна

Рецензент:
КОРОТУН Ольга Володимирівна

Житомир – 2026

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА КОМП'ЮТЕРНИХ НАУК

ЗАТВЕРДЖУЮ
В.о. завідувача кафедри комп'ютерних наук
Віталій ЛЕВКІВСЬКИЙ
«27» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу освітнього ступеня «бакалавр»
за спеціальністю 122 «Комп'ютерні науки»
(освітня програма «Комп'ютерна графіка та розробка ігор»)

Здобувач вищої освіти: **ТАЛЬКО Павло Вікторович**

Керівник роботи**: ПАНАРІНА Ірина Володимирівна**

Тема роботи: **«Розробка CRM системи для онлайн школи»,**

затверджена наказом закладу вищої освіти від **«27» лютого 2026 р., №95/с**

Термін здачі закінченої роботи: «10» червня 2026 р.

Вихідні дані роботи (зазначається предмет і об'єкт дослідження):

об'єкт дослідження є процес автоматизації управління навчальною діяльністю, клієнтською взаємодією та фінансовим моніторингом в установах позашкільної ІТ-освіти.

Консультанти випускної кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

Розділ	Консультант	Дата	
		Завдання видав	Завдання прийняв

Календарний план

№ з/п	Назва етапів випускної кваліфікаційної роботи	Термін виконання етапів роботи	Примітка
1	Пошук, огляд та аналіз аналогічних розробок. Постановка завдання. Опрацювання літературних джерел.	2 лютого 2026 – 5 березня 2026	Виконано
2	Проектування структури системи	6 березня 2026 – 25 березня 2026	Виконано
3	Написання програмного коду	26 березня 2026 – 26 квітня 2026	Виконано
4	Тестування системи	27 квітня 2026 – 14 травня 2026	Виконано
5	Оформлення пояснювальної записки	15 травня 2026 – 1 червня 2026	Виконано
6	Попередній захист	1 червня 2026	Виконано

Здобувач вищої освіти **Павло ТАЛЬКО**
Керівник **Ірина ПАНАРІНА**

РЕФЕРАТ

Основний зміст роботи викладено на 65 сторінках тексту, містить 4 таблиці, 26 ілюстрації, 4 додатки, налічує 30 найменувань літературних джерел.

Метою кваліфікаційної роботи є проєктування та розробка спеціалізованої веб-орієнтованої CRM-системи для IT-школи з метою автоматизації процесів управління навчанням, фінансового обліку та організації взаємодії між викладачами і студентами.

У кваліфікаційній роботі проведено системний аналіз предметної області освітніх організацій, проаналізував різні LMS та CRM-системи, що вже є на ринку, а також сформульовано постановку задачі на розробку власного програмного продукту. Для розробки було обрано, зокрема PHP-фреймворку Laravel, системи управління базами даних MySQL та сучасних інструментів фронтенд-розробки.

В результаті розроблено повнофункціональну CRM-систему із розширеною системою ролей (адміністратор, викладач, студент), алгоритмами автоматичної генерації чернеток розкладу, модулем перевірки домашніх завдань та механізмом динамічного контролю балансу оплачених занять користувача.

Ключовою технічною перевагою платформи є інтеграція зовнішніх сервісів: платіжного шлюзу WayForPay для еквайрингу та автоматичного поповнення балансу уроків, а також API сервісу відеоконференцій Zoom для автоматичного створення онлайн-кімнат під час проведення занять.

Проведено тестування розробленого програмного продукту, яке підтвердило повну відповідність платформи поставленим технічним вимогам та готовність до експлуатації в реальних умовах освітнього закладу.

Ключові слова: CRM-СИСТЕМА, LMS, LARAVEL, WAYFORPAY, ZOOM API, АВТОМАТИЗАЦІЯ ОСВІТНЬОГО ПРОЦЕСУ, РОЗКЛАД ЗАНЯТЬ, ДВОФАКТОРНА АВТЕНТИФІКАЦІЯ, ВЕБ-ПЛАТФОРМА.

ABSTRACT

The main content of the work is presented on 65 pages of text, contains 4 tables, 26 illustrations, 4 appendices, and includes 30 references.

The purpose of the qualification work is to design and develop a specialized web-based CRM system for an IT school aimed at automating educational process management, financial accounting, and organizing interaction between teachers and students.

In the qualification work, a system analysis of the educational organizations' subject area was performed, a comparative review of existing learning management systems (LMS) and CRM solutions was conducted, and the problem statement for developing a proprietary software product was formulated. The choice of development tools was justified, in particular, the Laravel PHP framework, the MySQL database management system, and modern frontend development tools.

As a result, a fully functional CRM system was developed, featuring an advanced role-based access control system (administrator, teacher, student), algorithms for automatic schedule draft generation, a homework review module, and a dynamic tracking mechanism for users' paid lesson balances.

The key technical advantage of the platform is the integration of external services: the WayForPay payment gateway for acquiring and automatically topping up lesson balances, as well as the Zoom video conferencing API for automatically creating online rooms for classes.

Functional and unit testing of the developed software product was conducted, confirming the platform's full compliance with the established technical requirements, the stability of its architectural components, and its readiness for operation in real educational institution conditions.

Keywords: CRM SYSTEM, LMS, LARAVEL, WAYFORPAY, ZOOM API, EDUCATIONAL PROCESS AUTOMATION, CLASS SCHEDULE, TWO-FACTOR AUTHENTICATION, WEB PLATFORM.

ЗМІСТ

ВСТУП 7
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ІСНУЮЧИХ РІШЕНЬ 9
1.1 Аналіз предметної області та характеристика об’єкту автоматизації 9
1.2 Огляд аналогів програмного застосунку 11
1.3 Обґрунтування концепції та постановка задачі на розробку спеціалізованої CRM-системи 16
1.4 Вибір технологій реалізації 18
Висновки до першого розділу 21
РОЗДІЛ 2. ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА БАЗИ ДАНИХ 23
2.1 Специфікація вимог та високорівневе моделювання системи 23
2.2 Проєктування архітектури серверної частини системи та інтеграцій 27
2.3 Проєктування логічної структури бази даних 31
2.4 Проєктування архітектури клієнтського застосунку та користувацьких інтерфейсів 36
Висновки до другого розділу 41
РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ CRM-СИСТЕМИ 42
3.1 Практична реалізація модулів серверної частини та інтеграція зовнішніх сервісів 42
3.2 Розробка клієнтського застосунку та опис кабінетів користувачів 47
3.3 Опис роботи з системою 53
3.4 Функціональне тестування програмного продукту 57
Висновки до третього розділу 63
ВИСНОВКИ 64
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ 66
ДОДАТКИ 68

ВСТУП

Актуальність теми. На сучасному етапі розвитку цифрової економіки та глобалізації ринку праці сфера IT-освіти демонструє стрімкі темпи зростання. Поява великої кількості приватних освітніх центрів, онлайн-шкіл та спеціалізованих IT-курсів створює високу конкуренцію, що вимагає від організаторів навчання забезпечення максимальної ефективності внутрішніх бізнес-процесів та бездоганної якості взаємодії зі студентами.

Традиційні методи обліку в освітніх закладах, що базуються на електронних таблицях, паперових документах чи універсальних системах, є малоефективними. Вони спричиняють помилки в розкладах, ускладнюють фінансовий контроль, перевірку завдань і комунікацію між викладачами та студентами. Наявні CRM-рішення часто потребують дорогого доопрацювання та не враховують специфіку освітнього процесу.

Отже, існує потреба у розробці спеціалізованої веб-орієнтованої CRM-системи, яка поєднує облік клієнтів, автоматизацію фінансових операцій та управління навчальним процесом (LMS). Використання Laravel, сучасних архітектурних підходів і API-інтеграцій для платежів та відеозв'язку є важливим кроком для модернізації освітньої інфраструктури.

Мета і завдання дослідження. Метою цієї роботи є проєктування, розробка та тестування високопродуктивної веб-платформи (CRM-системи) для автоматизації управління навчальним процесом, фінансового обліку та організації взаємодії між учасниками освітнього простору в межах школи IT-курсів.

Для досягнення поставленої мети необхідно вирішити такі завдання дослідження:

- виконати системний аналіз предметної області освітніх організацій та провести порівняльний аналіз існуючих аналогів інформаційних систем для управління навчанням;
- обґрунтувати вибір оптимального стека технологій та інструментальних засобів розробки серверної та клієнтської частин системи;

- спроєктувати трирівневу клієнт-серверну архітектуру застосунку, логічну модель реляційної бази даних та деталізувати ролі користувачів;
- розробити ядро серверної частини системи на базі фреймворку Laravel із реалізацією гнучкої системи розмежування прав доступу та механізму двофакторної автентифікації;
- програмно реалізувати бізнес-логіку модулів автоматичного формування чернеток розкладу, динамічного контролю грошового балансу студентів та пакетного обліку занять;
- інтегрувати зовнішні хмарні сервіси: API сервісу відеоконференцій Zoom для автоматичного створення онлайн-уроків та платіжний шлюз WayForPay для безпечного еквайрингу;
- провести комплексне функціональне, модульне та інтеграційне тестування розроблених компонентів програмного продукту.

Об‘єкт дослідження – процес автоматизації управління навчальною діяльністю, клієнтською взаємодією та фінансовим моніторингом в установах позашкільної ІТ-освіти.

Предметом дослідження – методи, архітектурні шаблони, алгоритми та інструментальні засоби проєктування, розробки та інтеграції спеціалізованих веб-орієнтованих CRM/LMS платформ.

Практичне значення отриманих результатів полягає у створенні готового до реальної експлуатації повнофункціонального програмного комплексу автоматизації ІТ-школи. Система забезпечує централізоване керування розкладом, автоматичне створення кімнат для занять, прийом онлайн-оплат з автоматичним нарахуванням балансу уроків клієнтам, а також надає зручні особисті кабінети для адміністраторів, викладачів та студентів із захистом персональних даних через механізми 2FA.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ІСНУЮЧИХ РІШЕНЬ

1.1 Аналіз предметної області та характеристика об’єкту автоматизації

На сучасному етапі розвитку цифрової економіки неформальна освіта, зокрема спеціалізовані IT-курси та онлайн-школи, демонструє безпрецедентні темпи зростання. Стрімка цифровізація суспільства вимагає нових підходів до організації освітнього простору, де ключову роль відіграють інформаційно-комунікаційні технології [1]. Однак разом із масштабуванням бізнесу, збільшенням кількості студентів та викладачів, керівництво освітніх закладів стикається з критичною проблемою управління великими масивами даних та неефективністю внутрішніх операційних процесів.

Дослідження показують, що традиційна модель управління освітнім центром, яка базується на використанні неспеціалізованого програмного забезпечення (наприклад, електронних таблиць Excel) та розрізнених месенджерів, швидко вичерпує свій потенціал при досягненні певного порогу кількості клієнтів [2]. Такий децентралізований підхід породжує явище «інформаційних бункерів» (data silos), коли дані про успішність, фінансовий стан та розклад студента знаходяться в різних, не пов'язаних між собою системах [3].

Зокрема, в контексті управління IT-школою можна виділити наступні критичні проблеми, що потребують автоматизації:

- складність фінансового обліку та контролю балансів. В освітніх закладах часто застосовується пакетна система оплати (наприклад, передплата за 10 занять). Ручний підрахунок відвіданих студентом уроків та залишку на його балансі неминуче призводить до помилок, викликаних людським фактором. За даними експертів з клієнтського менеджменту, відсутність автоматизованого білінгу є однією з головних причин фінансових втрат у сфері послуг [4];
- конфлікти при формуванні розкладів. Планування розкладів для десятків груп з урахуванням індивідуальної доступності викладачів та студентів є

складною комбінаторною задачею. Ручне керування цим процесом часто призводить до «накладок» або простоїв;

- деорганізація навчального процесу та перевірки знань. Відсутність єдиного простору (LMS – Learning Management System) для публікації матеріалів уроків та здачі домашніх завдань ускладнює процес навчання [5]. Файли губляться в корпоративних чатах, а відстеження прогресу студента стає неможливим, що прямо знижує якість надаваних освітніх послуг;
- високі витрати часу на адміністративну рутину. Організація онлайн-занять вимагає постійної генерації посилань для відеоконференцій (наприклад, у Zoom) та їх розсилки студентам. Ручне виконання цих рутинних операцій забирає до 30% робочого часу адміністраторів [2].

З огляду на ці виклики, об’єктом автоматизації в даній роботі виступає сукупність управлінських, фінансових та навчальних процесів типової IT-школи.

Для вирішення описаних проблем необхідно створити єдину інтегровану інформаційну екосистему, яка поєднуватиме функції CRM (Customer Relationship Management) для управління клієнтською базою та фінансами, а також LMS для управління безпосередньо контентом та навчанням. Відповідно до сучасних вимог до освітніх систем [3, 5], розроблюваний програмний продукт має забезпечити виконання наступних функцій:

- управління ролями та доступом: чітке розмежування прав між адміністраторами, викладачами та студентами (з можливістю контролю зі сторони батьків/опікунів);
- автоматизація планування: алгоритмічне створення чернеток розкладу на основі збережених даних про доступність викладачів (Teacher Availability);
- динамічний контроль фінансів: автоматичний розрахунок балансу занять для кожного студента та інтеграція платіжного шлюзу (WayForPay) для миттєвої онлайн-оплати тарифних планів;

- модуль управління навчанням: прикріплення навчальних матеріалів до уроків, видача домашніх завдань та інтерфейс для їх завантаження і перевірки (Homework Submissions);
- синхронізація з платформами відеозв'язку: інтеграція через API із сервісом Zoom для автоматичного створення кімнат під заплановані в розкладі заняття.

Результатом автоматизації стане комплексна веб-орієнтована платформа, яка мінімізує вплив людського фактора, підвищить прозорість фінансових потоків та створить високотехнологічне цифрове середовище, що відповідає сучасним стандартам IT-освіти.

1.2 Огляд аналогів програмного застосунку

На сучасному ринку програмного забезпечення для освітньої сфери представлено широкий спектр інструментів, які за останнє десятиліття еволюціонували від простих баз даних до складних хмарних екосистем. Згідно з дослідженнями освітніх технологій, ринок історично розділився на два ізольовані напрямки: системи управління навчанням (Learning Management Systems, LMS), що фокусуються на педагогічній складовій, та системи управління взаємовідносинами з клієнтами (Customer Relationship Management, CRM), що оптимізують комерційну діяльність [6].

Щоб обґрунтувати доцільність розробки власної гібридної CRM/LMS-системи для IT-школи, необхідно провести детальний огляд найпопулярніших рішень та здійснити їх глибокий функціональний аналіз у контексті реальних бізнес-процесів комерційного навчального закладу. Специфіка функціонування IT-школи полягає у тому, що система має не лише забезпечувати доставку навчального контенту, а й жорстко контролювати рентабельність, обробляти транзакції та керувати навантаженням викладачів.

Для дослідження було обрано три платформи, які найчастіше впроваджуються українськими освітніми центрами і представляють різні сегменти ринку: Moodle (сегмент академічних LMS), KeyCRM (сегмент

комерційних CRM для сфери послуг) та Google Classroom (сегмент базових хмарних інструментів).

Система управління навчанням Moodle. Moodle є світовим стандартом та найвідомішою LMS з відкритим вихідним кодом. Її архітектура розроблялася переважно для потреб вищих навчальних закладів та корпоративного навчання, де головний акцент робиться на довгостроковій академічній складовій [7].

Переваги: платформа пропонує вичерпний інструментарій для організації навчального процесу. Вона дозволяє створювати ієрархічні курси, формувати складні тести з автоматичною перевіркою, збирати домашні завдання у вигляді архівів чи посилань на репозиторії (що особливо актуально для IT-курсів) та вести детальний журнал успішності. Підтримується інтеграція інтерактивного контенту (стандарти SCORM та xAPI).

Недоліки (в контексті комерційної IT-школи): система абсолютно не пристосована для потреб малого та середнього бізнесу. У ній відсутні нативні модулі для білінгу та підрахунку залишку пакетних оплат (наприклад, коли клієнт купує абонемент на 10 занять, і система має автоматично списувати одне заняття після відвідування). Інтеграція українських платіжних шлюзів (WayForPay, LiqPay) вимагає залучення сторонніх розробників для написання дорогих кастомних плагінів. Крім того, інтерфейс системи є перевантаженим і морально застарілим, що вимагає тривалого онбордингу нових співробітників [8].

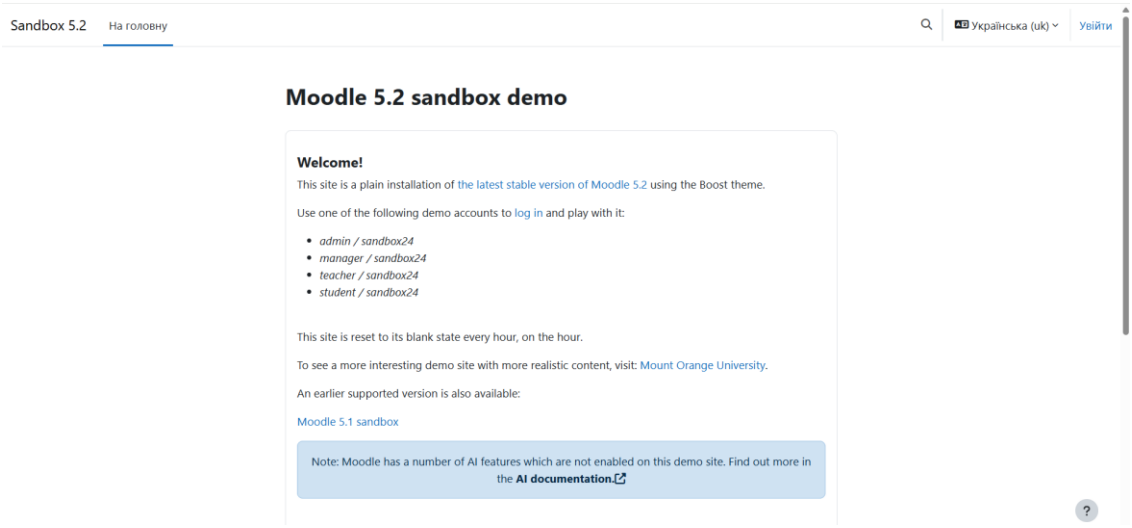


Рис. 1.1. Інтерфейс перегляду тестової сторінки курсу [8]

Клієнтська та фінансова система KeyCRM. KeyCRM — це сучасний український хмарний продукт (SaaS), розроблений для автоматизації процесів електронної комерції та сфери послуг. Завдяки високій гнучкості, цю систему часто намагаються адаптувати під освітні центри.

Переваги: платформа бездоганно справляється з фінансовим обліком та обробкою бази клієнтів. Вона має потужні інструменти для ведення воронок продажів (Канбан-дошки), контролю оплат, створення інвойсів та підтримки прямої інтеграції з українськими банками (Monobank, PrivatBank) і месенджерами (Telegram, Viber). Інтерфейс відрізняється високою реактивністю, адаптивністю під мобільні пристрої та ергономічністю [9].

Недоліки (в контексті комерційної IT-школи): це виключно бізнес-інструмент, який не має педагогічного модуля (LMS-складової). У KeyCRM неможливо створити повноцінний особистий кабінет студента, де він міг би переглядати відеозаписи лекцій, завантажувати файли з виконаними практичними роботами або отримувати текстовий зворотний зв'язок від викладача щодо написаного коду. Також відсутні механізми генерації розкладів груп та автоматичного створення Zoom-кімнат на основі розкладу.

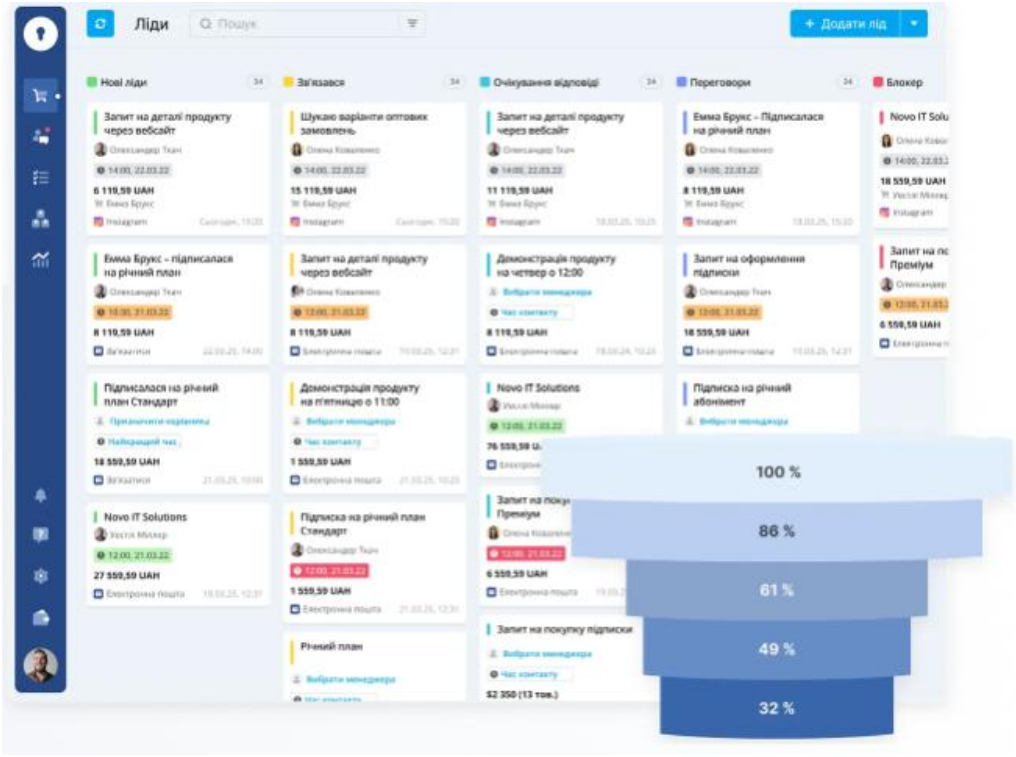


Рис. 1.2. Інтерфейс управління клієнтською базою та фінансами в KeyCRM

Хмарний сервіс Google Classroom. Google Classroom — це базовий безкоштовний сервіс, що входить до екосистеми Google Workspace. Він розроблений переважно для закладів загальної середньої освіти та індивідуальних репетиторів [10].

Переваги: сервіс є надзвичайно простим у використанні, не потребує розгортання на власних серверах та інтуїтивно зрозумілий. Він дозволяє швидко публікувати навчальні матеріали, збирати файли та залишати коментарі до робіт. Ключовою перевагою є безшовна інтеграція з іншими сервісами Google, зокрема з Google Meet для проведення онлайн-занять та Google Drive для зберігання файлів.

Недоліки (в контексті комерційної IT-школи): платформа повністю позбавлена інструментів для комерційного адміністрування. Відсутні будь-які можливості моніторингу заборгованостей студентів. Крім того, система ролей є жорстко обмеженою: існують лише профілі «викладач» і «студент». Це унеможливлює створення ієрархічних акаунтів для адміністраторів чи менеджерів школи, які повинні мати доступ до фінансів та загального розкладу, але не втручатися в навчальний процес.

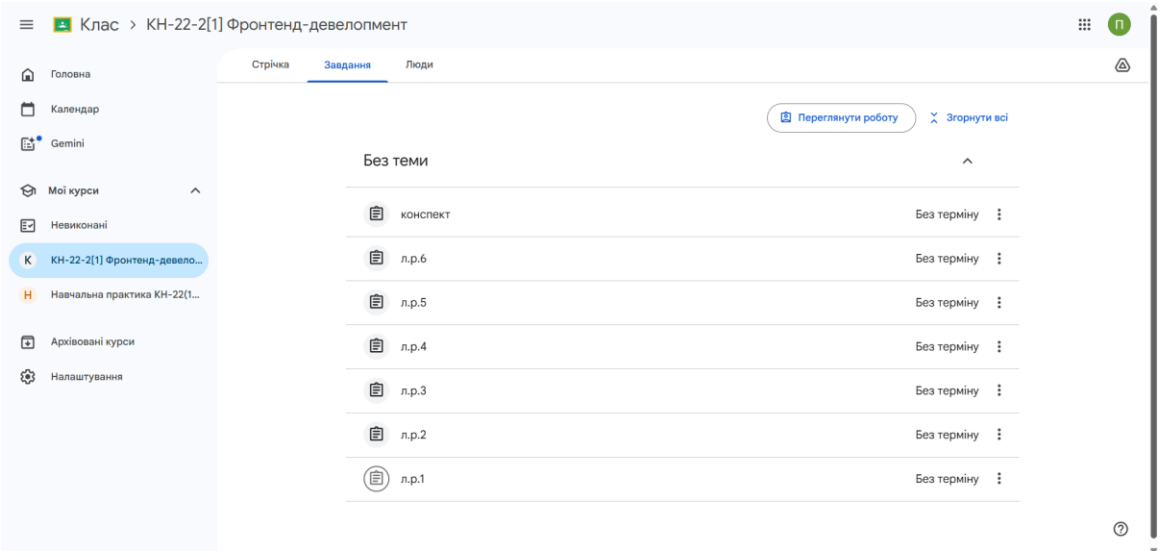


Рис. 1.3. Організація навчальних матеріалів у сервісі Google Classroom

Для наочної систематизації виявлених характеристик та визначення здатності кожної системи задовольняти комплексні потреби IT-школи було

сформовано описову матрицю порівняння. Оцінювання здійснювалося за ключовими бізнес-модулями (табл.1.1).

Таблиця 1.1

Функціональний порівняльний аналіз існуючих платформ

Назва модуля / Функціональна вимога	**Moodle (Академічна LMS)**	**KeyCRM (Комерційна CRM)**	**Google Classroom (Базова LMS)**
Фінансовий облік та білінг (Баланс занять, інтеграція платіжних шлюзів)	Відсутній. Не підтримує пакетні оплати занять. Потребує залучення розробників для написання дорогих кастомних плагінів.	Високий рівень. Вбудована воронка оплат, пряма інтеграція з банками, зручний контроль заборгованостей та аналітика.	Відсутній. Повністю безкоштовна платформа без можливості ведення будь-якого комерційного обліку.
Навчальний процес (LMS) (Кабінети студентів, завантаження домашніх завдань)	Високий рівень. Повний контроль над контентом, потужна система тестування, інтеграція зовнішніх сховищ та гнучкий журнал оцінок.	Відсутній. Немає кабінетів для студентів, неможливо прикріплювати навчальні матеріали, відеозаписи чи збирати код.	Середній рівень. Зручна видача завдань та збір файлів, але без гнучкої аналітики успішності та профілів навичок.
Керування розкладом (Матриці доступності викладачів, генерація Zoom-посилань)	Частково. Можлива синхронізація із Zoom через складні налаштування API, проте немає генерації розкладу за доступністю.	Відсутня. Система фіксує лише події в календарі менеджера, не працює з групами та віртуальними кімнатами.	Базовий рівень. Існує прив'язка лише до сервісу Google Meet, відсутні алгоритми врахування вільного часу викладача.
Ролі та безпека доступу (Розмежування прав, 2FA)	Надмірно складна. Дозволяє налаштувати будь-які права на рівні системного адміністратора, але процес є вкрай недружнім до користувача.	Гнучка для бізнесу. Орієнтована виключно на менеджерський склад. Не передбачає ролі «Студент» із доступом до матеріалів.	Обмежена. Лише дві базові ролі: Викладач та Учень. Комплексне адміністрування школи (менеджмент) не передбачено.

Проведений порівняльний аналіз яскраво демонструє фрагментарність сучасного ринку освітнього програмного забезпечення. Існуючі системи жорстко сегментовані: вони здатні ефективно вирішувати або завдання організації

навчання (Moodle, Google Classroom), або завдання бізнес-обліку та продажу (KeyCRM) [6, 9].

Універсального "коробкового" рішення, яке б органічно поєднувало автоматизацію перевірки домашніх завдань зі списанням уроків з персонального грошового балансу клієнта та нативною генерацією Zoom-посилань, серед розглянутих базових продуктів немає. На практиці це змушує освітні центри використовувати кілька окремих платформ одночасно та налаштовувати їхню взаємодію через сторонні сервіси-інтегратори (наприклад, Zapier). Такий підхід не лише збільшує постійні операційні витрати школи на підписки (SaaS-податок), але й створює високі ризики втрати даних через збої в API-синхронізації. Виявлена проблематика повністю підтверджує технічну та економічну необхідність розробки власного гібридного програмного продукту, архітектура якого від самого початку враховуватиме специфіку IT-школи.

1.3 Обґрунтування концепції та постановка задачі на розробку спеціалізованої CRM-системи

Проведений у попередньому підрозділі функціональний аналіз ринку програмного забезпечення виявив суттєву системну прогалину. Більшість освітніх закладів змушені йти на компроміс: або використовувати кілька ізольованих програм (що призводить до дублювання даних та фінансових втрат), або вкладати значні кошти у тривале доопрацювання універсальних CRM-систем під свою вузьку специфіку. Відповідно до сучасних методологій інженерії програмного забезпечення, за наявності таких обмежень готових (Off-the-Shelf) продуктів, найбільш раціональним рішенням є розробка власного (Custom) програмного забезпечення, архітектура якого від початку відповідатиме бізнес-логіці конкретної організації [11].

Зважаючи на це, концепція розробки власної спеціалізованої веб-орієнтованої платформи для IT-школи є технічно обґрунтованою та комерційно доцільною. Головною ідеєю проєкту є створення безшовного гібридного цифрового середовища (CRM + LMS), де весь життєвий цикл студента — від моменту реєстрації, купівлі тарифного плану до відвідування онлайн-занять та

перевірки випускного коду — супроводжується автоматизованими алгоритмами (рис. 1.4).

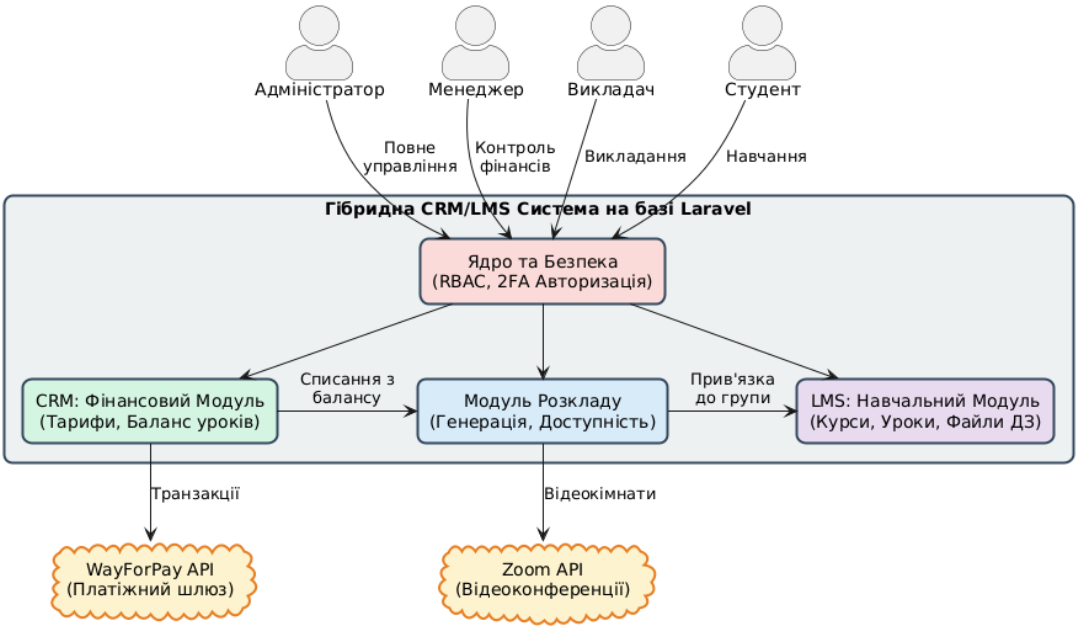


Рис. 1.4. Концептуальна модель гібридної архітектури CRM/LMS-системи

В основу концепції покладено використання сучасного паттерну проєктування MVC (Model-View-Controller) та архітектурних шарів Service Layer / Repository Pattern, що забезпечить високу масштабованість системи у майбутньому [12]. Як базовий інструментарій доцільно обрати PHP-фреймворк Laravel, оскільки він має вбудовані надійні механізми маршрутизації, ORM-систему Eloquent для роботи з базами даних та широкі можливості для інтеграції зовнішніх API.

Відповідно до концепції, метою даної кваліфікаційної роботи є проєктування та практична програмна реалізація веб-орієнтованої CRM-системи для комплексної автоматизації управління навчальним процесом та фінансовим обліком IT-школи.

Для досягнення поставленої мети сформульовано наступні функціональні та архітектурні завдання (постановка задачі на розробку):

– проєктування бази даних: розробити логічну структуру реляційної бази даних, яка забезпечить надійне зберігання зв'язків між сутностями системи: користувачами (Users), групами (Groups), розкладами (Schedules), платежами (Payments) та домашніми завданнями (Homework Submissions);

- реалізація системи авторизації та ролей: створити ядро безпеки застосунку із жорстким розмежуванням прав доступу на рівні Middleware для трьох основних ролей: «Адміністратор», «Викладач», «Студент». Додатково впровадити алгоритм двофакторної автентифікації (2FA) для захисту облікових записів [13];
- розробка фінансового модуля: програмно реалізувати бізнес-логіку пакетних оплат, динамічного підрахунку балансу уроків клієнта та інтегрувати зовнішній платіжний шлюз WayForPay для автоматичної обробки транзакцій;
- автоматизація планування розкладів: розробити алгоритмічний модуль, який збиратиме дані про вільні години викладачів та студентів і на їхній основі генеруватиме чернетки розкладів занять;
- інтеграція відеозв'язку: реалізувати зв'язок серверної частини CRM із зовнішнім API сервісу Zoom для автоматичного генерування посилань на відеоконференції безпосередньо в момент затвердження розкладу [14];
- розробка навчального простору: створити інтерфейси для публікації навчального контенту, видачі завдань студентам, а також інструментарій для завантаження студентами файлів із кодом і їх подальшого оцінювання викладачами;
- тестування та валідація: покрити ключову бізнес-логіку системи (особливо алгоритми списання балансу та генерації розкладів) автоматизованими модульними (Unit) та функціональними (Feature) тестами для підтвердження стабільності програмного коду.

1.4 Вибір технологій реалізації

Етап програмної реалізації вимагає ретельного підбору інструментальних засобів, бібліотек та фреймворків, які здатні забезпечити виконання усіх архітектурних вимог, визначених у попередньому розділі. Оскільки розроблювана гібридна CRM/LMS-система призначена для автоматизації бізнес-процесів IT-школи (фінансовий облік, управління розкладом та оцінювання

студентів), технологічний стек повинен гарантувати високу безпеку транзакцій, швидкість розробки та легкість подальшого масштабування [15].

В якості фундаменту для бекенд-розробки було обрано мову програмування PHP (версії 8.1+) та екосистему фреймворку Laravel (версії 10). Вибір саме цього фреймворку обґрунтовано наявністю потужних вбудованих компонентів, які покривають більшість потреб корпоративної розробки [16]:

- об'єктно-реляційне відображення (ORM Eloquent): забезпечує взаємодію з базою даних через об'єктно-орієнтований інтерфейс (Active Record). Це усуває необхідність написання "сирих" SQL-запитів і забезпечує захист від SQL-ін'єкцій через використання підготовлених виразів (Prepared Statements) під капотом;
- маршрутизація та Middleware: Laravel надає гнучку систему маршрутизації (routes/web.php, routes/api.php). Для захисту маршрутів було використано проміжне програмне забезпечення. Як видно з кодової бази, у проєкті реалізовано власні фільтри: RoleMiddleware.php для розмежування доступу між адміністраторами, викладачами та студентами, а також TwoFactorMiddleware.php для перевірки статусу 2FA;
- контейнер впровадження залежностей (IoC Container): для забезпечення архітектури Service-Repository, в ядрі системи був створений кастомний сервіс-провайдер app/Providers/RepositoryServiceProvider.php. Він відповідає за зв'язування інтерфейсів репозиторіїв (наприклад, CourseRepositoryInterface) з їхніми конкретними реалізаціями (CourseRepository), що є еталонним прикладом дотримання принципу Inversion of Control;
- консольний інструментарій (Artisan): для автоматизації фонових задач було розроблено власні консольні команди, зокрема app/Console/Commands/GenerateDraftSchedules.php. Використання планувальника задач (app/Console/Kernel.php) дозволяє системі автономно генерувати розклади та створювати Zoom-кімнати без участі людини.

Враховуючи роботу з фінансами та персональними даними, система вимагає підвищеного рівня захисту. Стандартна автентифікація Laravel була розширена модулем двофакторної автентифікації. Для цього в проєкт інтегровано бібліотеку Google2FA (наявний конфігураційний файл config/google2fa.php та контролер TwoFactorController.php). Цей інструмент генерує секретні ключі на основі алгоритму TOTP (Time-based One-Time Password, стандарт RFC 6238) та дозволяє користувачам захищати свої кабінети за допомогою застосунку Google Authenticator [17].

Інструментарій клієнтської частини спроєктовано за принципом серверного рендерингу з мінімальним використанням важких JavaScript-фреймворків:

- шаблонізатор Blade: використовується для динамічної генерації HTML-розмітки на стороні сервера. Blade дозволяє використовувати контрольні структури безпосередньо у представленнях та розбивати інтерфейс на компоненти (наприклад, <x-page-header>);
- Tailwind CSS: сучасний утилітарний CSS-фреймворк, обраний для стилізації інтерфейсів [18]. Використання Tailwind дозволяє створювати адаптивний дизайн безпосередньо у класах HTML-елементів (у проєкті наявний шаблон resources/views/vendor/pagination/tailwind.blade.php), що пришвидшує розробку та зменшує вагу кінцевого файлу стилів;
- FontAwesome: для забезпечення користувацького інтерфейсу інтуїтивно зрозумілою іконографікою використано бібліотеку FontAwesome 6.5, яка також імпортується через CDN.

Для розширення функціоналу системи залучено зовнішні API (Application Programming Interfaces):

- WayForPay API: використовується для організації платіжного шлюзу, прийому платежів за тарифи та автоматизації поповнення балансу занять (обробка вебхуків у WayForPayController.php);
- Zoom API (Server-to-Server OAuth): інструмент для автоматичного генерування посилань на відеоконференції під час створення розкладу;

– PHPUnit (phpunit.xml): інструмент для забезпечення якості коду [19]. У директорії tests/ створено набори модульних (Unit) та функціональних (Feature) тестів (наприклад, LessonBalanceServiceTest.php, WayForPayServiceTest.php), які автоматично перевіряють критичну бізнес-логіку системи перед розгортанням.

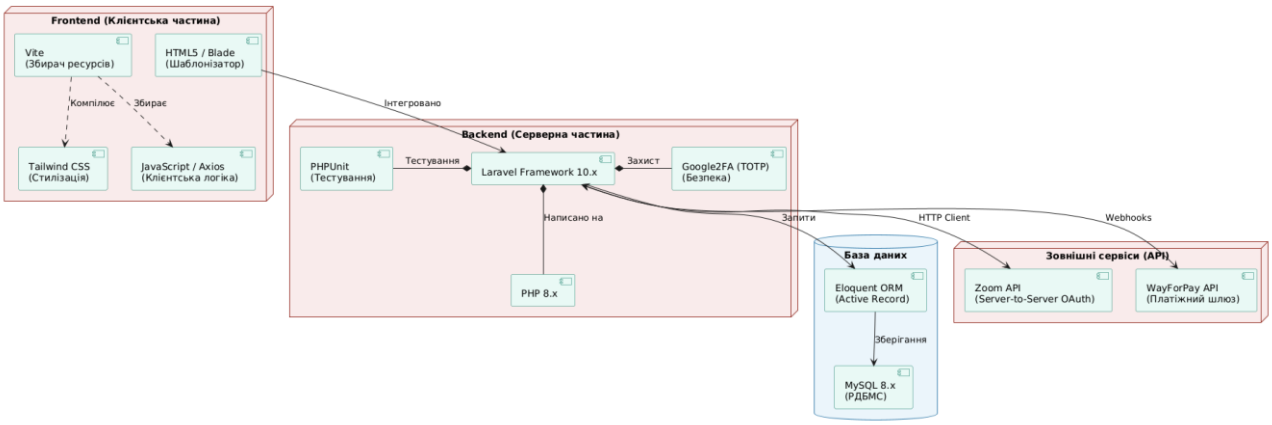


Рис.1.5. Архітектура технологічного стеку гібридної CRM/LMS-системи

Вибраний стек технологій є сучасним, збалансованим та повністю відповідає вимогам, поставленим на етапі проєктування. Використання Laravel у зв'язці з надійними інструментами безпеки забезпечує створення стабільного програмного продукту, детальна реалізація якого описується у наступному підрозділі.

Висновки до першого розділу

У першому розділі кваліфікаційної роботи було проведено детальний аналіз предметної області функціонування закладів неформальної IT-освіти, що дозволило виявити ключові особливості організації їхньої діяльності, основні бізнес-процеси та проблеми, які виникають у процесі управління навчальним середовищем. Особливу увагу було приділено питанням організації взаємодії між адміністраторами, викладачами та студентами, а також управлінню фінансовими та навчальними процесами.

У ході дослідження встановлено, що більшість сучасних приватних IT-шкіл використовують розрізнені інструменти для ведення обліку, зокрема електронні таблиці, месенджери, поштові сервіси та окремі хмарні платформи.

Такий підхід створює так званий ефект «інформаційних бункерів», коли важлива інформація зберігається у різних системах без централізованого доступу. Це значно ускладнює координацію роботи персоналу, підвищує ризик втрати даних, створює дублювання інформації та призводить до виникнення помилок у плануванні занять.

Було визначено, що особливо критичними є проблеми, пов’язані з фінансовим обліком. Зокрема, складність контролю балансів пакетних занять, відстеження оплат, нарахування залишків уроків та фіксація заборгованостей часто виконуються вручну, що підвищує ймовірність фінансових помилок та втрати прибутку. Крім того, ручне управління розкладом занять без урахування доступності викладачів, зайнятості аудиторій або онлайн-кімнат створює конфлікти у графіках та негативно впливає на якість навчального процесу.

На основі проведеного аналізу було обґрунтовано доцільність розробки власного спеціалізованого програмного забезпечення у вигляді гібридної CRM/LMS-системи. Як технологічну основу обрано Laravel, який забезпечує високий рівень надійності, масштабованості, безпеки та зручності реалізації складної бізнес-логіки. Запропонована система повинна включати реляційну базу даних для централізованого зберігання інформації, модуль автоматизованого фінансового білінгу, алгоритмічну систему формування розкладів, навчальний простір для обміну файлами, домашніми завданнями та результатами перевірки, а також механізми захисту персональних даних на основі двофакторної автентифікації (2FA) і рольової моделі доступу (RBAC).

РОЗДІЛ 2. ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА БАЗИ ДАНИХ

2.1 Специфікація вимог та високорівневе моделювання системи

Перехід від етапу системного аналізу до проєктування програмного забезпечення вимагає суворої формалізації вимог до майбутнього продукту. Специфікація вимог є фундаментальним кроком, що визначає межі функціональності розроблюваної гібридної CRM/LMS-системи для IT-школи та слугує основою для побудови архітектури серверної частини і логічної структури бази даних.

На основі аналізу бізнес-процесів освітнього закладу, розроблювана веб-орієнтована система повинна задовольняти наступні критичні функціональні вимоги, які логічно поділяються на чотири модулі:

- модуль безпеки та керування доступом: підтримка ієрархічної системи ролей (Role-Based Access Control) за допомогою проміжного програмного забезпечення (Middleware) для ізоляції запитів. Обов'язкова підтримка двофакторної автентифікації (2FA) для захисту облікових записів;
- фінансово-аналітичний модуль (CRM): створення та управління тарифними планами (TariffPlans) з визначенням кількості уроків у пакеті та їхньої вартості. Інтеграція платіжного шлюзу (WayForPay) для обробки онлайн-транзакцій з автоматичним збереженням логів платежів (Payments). Динамічний контроль балансу занять: автоматичний розрахунок залишку оплачених уроків клієнта та блокування доступу до навчального контенту при нульовому балансі;
- модуль управління навчанням (LMS): організація процесу перевірки знань: система повинна дозволяти викладачу прикріплювати файли із завданнями, студенту — завантажувати розв'язки, а викладачу — повертати перевірений код із коментарями (HomeworkSubmissions);
- модуль планування та комунікації: збір даних про вільні години викладачів та студентів через спеціальні матриці доступності (Availabilities). Автоматична генерація чернеток розкладу (Schedules) на основі вільного

часу. Інтеграція з API сервісу відеоконференцій (Zoom) для автоматичного генерування посилань на онлайн-уроки.

До нефункціональних вимог належать: висока продуктивність обробки HTTP-запитів, дотримання принципів RESTful API для потенційного масштабування, стійкість до вразливостей (SQL Injection, CSRF, XSS) та використання реляційної системи керування базами даних (MySQL) для забезпечення цілісності транзакцій (ACID).

Для формалізації вимог та визначення меж системи було використано методологію об'єктно-орієнтованого моделювання UML. Першим етапом є ідентифікація акторів — ролей, які взаємодіють із платформою. У розроблюваній базі даних визначено 4 ключові ролі:

- «Адміністратор» (Admin): системний користувач із повним доступом до налаштувань, фінансів та загального розкладу;
- «Менеджер» (Manager): контролююча роль, прив'язана до профілю студента (через guardian_fields), яка має право переглядати баланс та здійснювати оплати;
- «Викладач» (Teacher): користувач академічного блоку, який взаємодіє з розкладом та перевіряє домашні завдання;
- «Студент» (Student): кінцевий споживач освітніх послуг, який навчається та завантажує результати своєї роботи.

Було сформовано прецеденти (Use Cases) для кожного з акторів у текстовому представленні. Щоб у них легше було орієнтуватися під час подальшого проєктування системи, кожному з прецедентів присвоєно унікальний порядковий номер:

Для «Адміністратора»:

- [UC1] налаштування системних інтеграцій (Zoom, WayForPay);
- [UC2] створення та редагування тарифних планів;
- [UC3] генерація чернеток розкладу занять;
- [UC4] керування обліковими записами користувачів та ролями.

Для «Менеджера»:

- [UC5] моніторинг фінансових транзакцій та оплат;
- [UC6] контроль залишку (балансу) занять студента;
- [UC7] перегляд розкладу та загальної успішності студента.

Для «Викладача»:

- [UC8] налаштування матриці доступності (вільних годин);
- [UC9] проведення онлайн-уроків (підключення до кімнати Zoom);
- [UC10] публікація навчальних матеріалів до уроків;
- [UC11] перевірка виконаних домашніх завдань та відправка фідбеку.

Для «Студента»:

- [UC12] перегляд особистого розкладу та долучення до уроку;
- [UC13] завантаження файлів із виконаним домашнім завданням;
- [UC14] оплата тарифного плану для поповнення балансу занять;
- [UC15] налаштування безпеки профілю (двофакторна автентифікація 2FA).

Детально визначені прецеденти та сформована специфікація функціональних вимог не лише описують поведінку системи з точки зору кінцевих користувачів, але й слугують фундаментальною методологічною основою для подальших етапів розробки. Цей перелік діє як сполучна ланка між бізнес-правилами освітнього процесу та їх суто технічною реалізацією. Описані Use Cases дозволяють чітко окреслити межі відповідальності кожного модуля системи, мінімізувати ризики надмірної зв'язності коду та забезпечити високий рівень масштабованості майбутньої платформи.

Зокрема, виділені сценарії взаємодії формують жорсткі вимоги до проєктування багаторівневої програмної архітектури серверної частини. Оскільки система передбачає складні алгоритми обробки інформації (автоматичне списання занять з балансу, генерація розкладів з урахуванням матриць доступності, обробка статусів платежів через вебхуки), виникає об'єктивна необхідність архітектурного розподілу обов'язків. Уся ключова

бізнес-логіка, що випливає з описаних прецедентів, буде виділена в ізольований рівень сервісів (Service Layer). Такий підхід дозволить максимально розвантажити контролери, залишивши їм виключно функції прийому HTTP-запитів, базової валідації вхідних даних та форматування відповідей API.

Для наочної візуалізації описаних функціональних вимог, розуміння архітектури доступу (RBAC) та відображення меж розроблюваної системи було побудовано загальну UML-діаграму прецедентів (рис. 2.1).

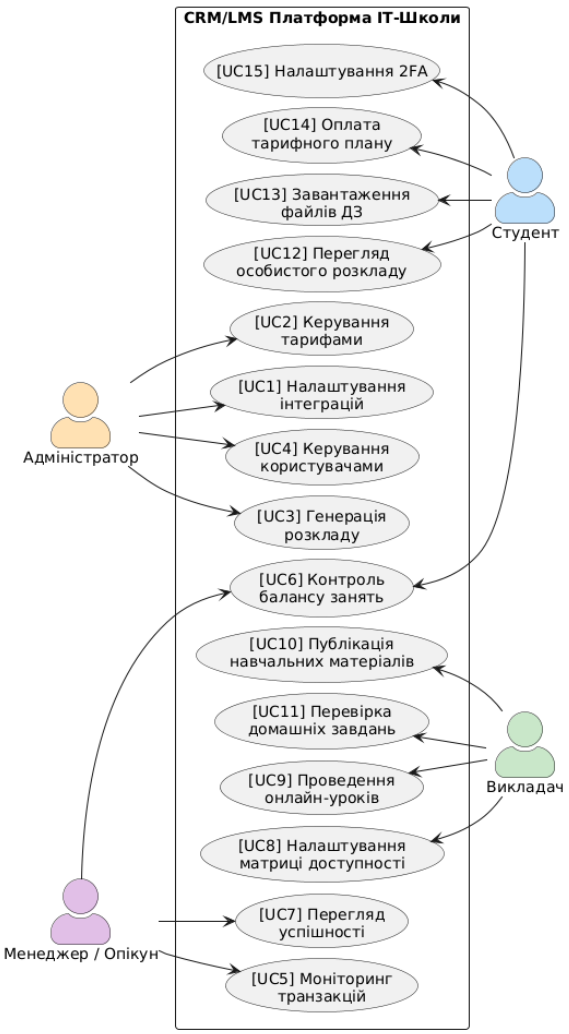


Рис. 2.1. UML-діаграма прецедентів взаємодії користувачів із CRM/LMS системою

2.2 Проєктування архітектури серверної частини системи та інтеграцій

Продуктивність, безпека та здатність до подальшого масштабування програмного продукту безпосередньо залежать від якості архітектури його серверної частини (бекенду).

Для вирішення проблеми зв'язності коду та забезпечення його гнучкості, архітектуру серверної частини було розширено до багаторівневої (N-Tier Architecture). У ядро системи було інтегровано патерни Service Layer (Шар сервісів) та Repository (Репозиторій).

Логіка обробки даних у розробленій системі розділена на три суворо ізольовані рівні:

- HTTP-Шар (Controllers): контролери (наприклад, PaymentController, ScheduleController) виконують роль тонких координаторів. Вони відповідають виключно за прийом вхідного HTTP-запиту, валідацію вхідних даних та повернення відповіді (View або JSON) клієнту. Контролер не містить математичних розрахунків і не звертається до бази даних напряму;
- шар бізнес-логіки (Services): усі складні алгоритми інкапсульовані у відповідних сервісних класах. Наприклад, LessonBalanceService відповідає за перевірку наявності доступних занять у студента, списує їх або нараховує нові. WayForPayService містить логіку криптографічного шифрування та комунікації з платіжним шлюзом. Такий підхід дозволяє повторно використовувати одну й ту саму бізнес-логіку як для веб-інтерфейсу, так і для фонових cron-задач або потенційного мобільного API;
- шар доступу до даних (Repositories): репозиторії (наприклад, CourseRepository, UserRepository) створюють абстракцію над базою даних. Вони містять усі SQL-запити. Завдяки цьому шару бізнес-логіка не знає, як саме дані зберігаються фізично, що відповідає принципу інверсії залежностей (Dependency Inversion Principle).

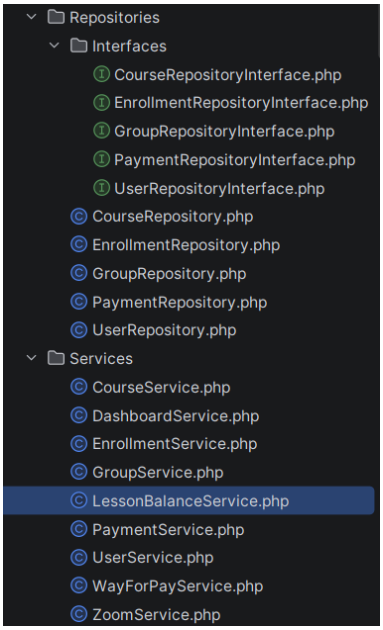


Рис. 2.2. Ієрархія каталогів розробленої системи з реалізацією патернів Service та Repository

Зв'язування цих шарів відбувається через механізм впровадження залежностей (Dependency Injection), який реалізується завдяки Service Container фреймворку Laravel. Замість того, щоб контролер самостійно створював екземпляр сервісу через оператор new, він оголошує залежність у своєму конструкторі, а фреймворк автоматично її «ін'єктує».

Нижче наведено лістинг коду, що ілюструє використання впровадження залежностей у контролері PaymentController:

```
namespace App\Http\Controllers;

use App\Models\Enrollment;
use App\Models\Payment;
use App\Services\LessonBalanceService;
use App\Services\PaymentService;
use App\Services\UserService;
use Illuminate\Http\Request;

class PaymentController extends Controller
{
    public function __construct(
        protected PaymentService $paymentService,
        protected UserService $userService,
        protected LessonBalanceService $balance
    ) {}

    public function index(Request $request)
    {
```

```
        $filters  = $request->only(['student_id', 'status',
'method']);
        $payments = $this->paymentService->getList($filters);
        return view('payments.index', compact('payments',
'filters'));
    }

    public function create()
    {
        $students    = $this->userService->getStudents();
        $enrollments = Enrollment::with(['student',
'group.course'])->get();
        $methods     = Payment::METHODS;
        $tariffPlans = \App\Models\TariffPlan::where('status',
'active')->get();
        return view('payments.create', compact('students',
'enrollments', 'methods', 'tariffPlans'));
    }
}
```

Для візуалізації інформаційних потоків між описаними архітектурними шарами було розроблено UML-діаграму компонентів (рис. 2.3).

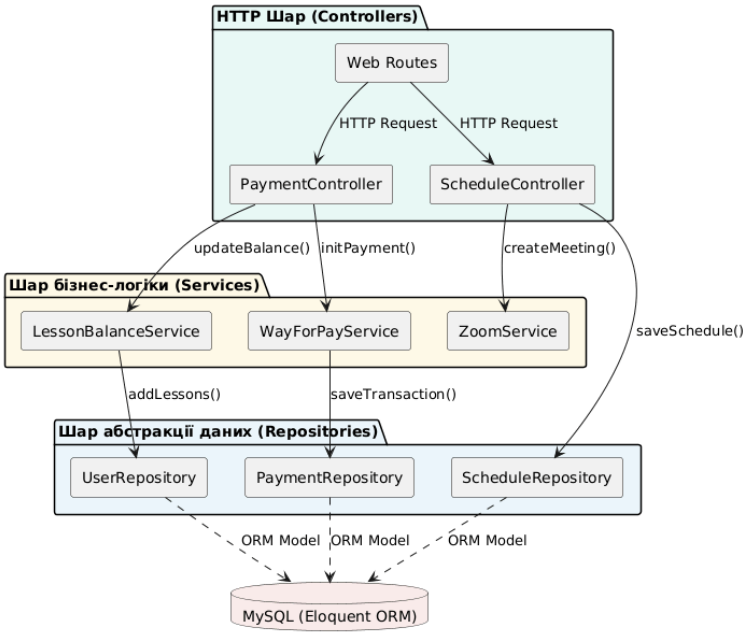


Рис. 2.3. UML-діаграма компонентів серверної частини

Ключовим бізнес-процесом комерційної ІТ-школи є автоматизація продажу тарифних планів (TariffPlans). Для реалізації цього процесу було спроєктовано інтеграцію з українським еквайринговим сервісом WayForPay. Враховуючи роботу з фінансами, архітектура цього модуля вимагає високого рівня криптографічного захисту транзакцій від можливих атак типу "Man-in-the-Middle" (MITM) та підміни даних клієнтом.

Процес обробки платежу реалізовано у два етапи:

– ініціалізація та криптографічний підпис: коли студент обирає тариф, WayForPayService генерує унікальний ідентифікатор транзакції (orderReference). Формується масив даних (ідентифікатор мерчанта, сума, валюта, назва тарифу). Відповідно до вимог WayForPay, цей масив конкатенується у єдиний рядок, після чого генерується хеш за алгоритмом HMAC_MD5 з використанням секретного ключа (SecretKey), який зберігається у захищеному файлі оточення .env. Згенерований підпис передається разом із формою, що гарантує цілісність даних (якщо користувач спробує змінити суму через інструменти розробника в браузері, хеш не співпаде, і банк відхилить транзакцію);

– асинхронна обробка Webhook-сповіщень: після успішного списання коштів банк не покладається на браузер користувача, а здійснює прямий Server-to-Server POST-запит (webhook) на захищений маршрут системи. Спеціальний метод сервісу валідує зворотний підпис банку. У разі успіху, запис у таблиці payments переводиться у статус «Approved», і викликається LessonBalanceService, який автоматично додає придбану кількість уроків до поля lessons_count у профілі користувача.

Для опису часової послідовності та взаємодії між системою, користувачем та банком було побудовано UML-діаграму послідовності (рис. 2.4).

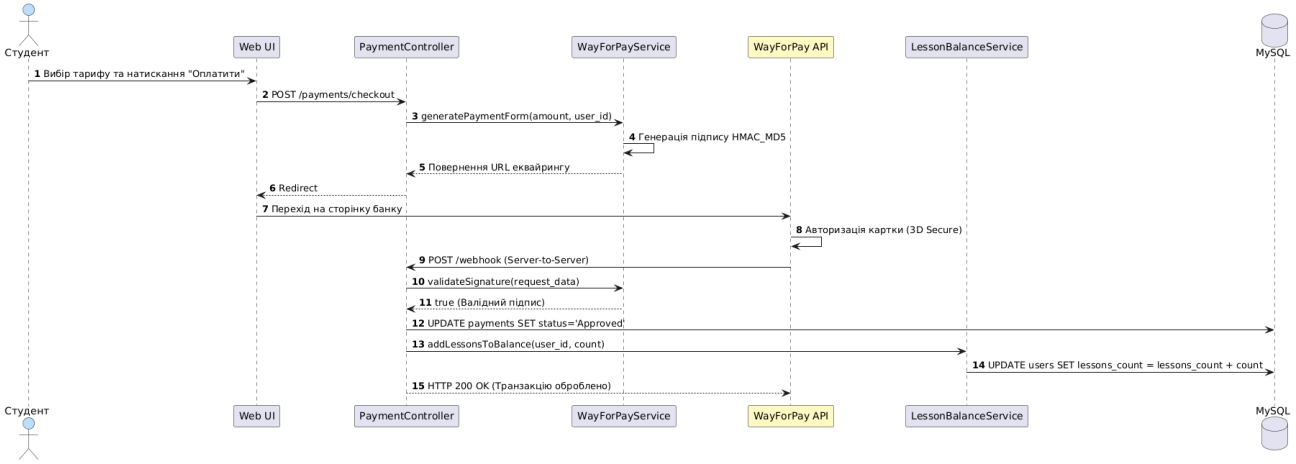


Рис. 2.4. UML-діаграма послідовності обробки платежу та оновлення балансу

Для проведення дистанційних занять архітектура передбачає глибоку інтеграцію з API сервісу відеоконференцій Zoom. Замість ручної генерації посилань, процес повністю автоматизовано за допомогою консольних команд Laravel (Artisan Commands) та планувальника задач (Task Scheduler).

Взаємодія із Zoom реалізується за протоколом Server-to-Server OAuth, що дозволяє бекенду CRM-системи самостійно отримувати токени доступу без авторизації користувача в браузері.

Алгоритм роботи модуля розкладу:

- у ядрі системи (app/Console/Kernel.php) зареєстровано команду GenerateDraftSchedules, яка запускається за розкладом (наприклад, щонеділі вночі);
- система аналізує таблицю TeacherAvailability (вільні вікна викладачів) та зіставляє їх із розкладом груп (Group);
- для кожного знайденого співпадіння ZoomService використовує клієнт Illuminate\Support\Facades\Http для здійснення POST-запиту до кінцевої точки api.zoom.us/v2/users/me/meetings;
- отримавши JSON-відповідь, система витягує з неї два критично важливих посилання: join_url (загальне посилання для студентів) та start_url (посилання для викладача з правами хоста);
- ці дані зберігаються в таблицю schedules, після чого розклад стає доступним в особистих кабінетах.

Використання багаторівневої архітектури у поєднанні з API зовнішніх платформ дозволило створити повністю автономне ядро системи, що не потребує втручання адміністратора для обробки рутинних операцій.

2.3 Проєктування логічної структури бази даних

Основою будь-якої інформаційної керуючої системи є надійна, масштабована та оптимізована база даних. Враховуючи гібридний характер розроблюваної платформи (поєднання функцій клієнтського фінансового обліку CRM та академічного управління LMS), до системи зберігання даних

висуваються підвищені вимоги щодо цілісності транзакцій, узгодженості та швидкодії.

Для реалізації фізичної моделі даних було обрано реляційну систему керування базами даних (РСКБД) MySQL. Цей вибір обґрунтований повною підтримкою властивостей ACID (Atomicity, Consistency, Isolation, Durability), що є критично важливим для обробки фінансових платежів через WayForPay та ведення точного балансу оплачених занять клієнтів. Взаємодія серверної логіки з базою даних здійснюється через патерн Active Record за допомогою ORM-системи Eloquent, яка вбудована у фреймворк Laravel.

Для уникнення надмірності даних (дублювання) та запобігання аномаліям оновлення, видалення і вставки, логічну структуру бази даних було спроєктовано з дотриманням правил Третьої нормальної форми (3NF). Усі неключові атрибути кожної таблиці повністю і транзитивно залежать виключно від її первинного ключа (Primary Key).

На основі специфікації вимог, сформованої у підрозділі 2.1, було виділено наступні ключові домени (групи сутностей) та відповідні їм таблиці:

1. Домен ідентифікації та керування доступом (Identity & Access Domain):
   - users — центральна таблиця системи, що зберігає облікові записи. Окрім стандартних полів аутентифікації, таблиця містить поле role для реалізації RBAC («Адміністратор», «Викладач», «Студент»), поле lessons_count (грошово-урочний баланс студента), а також групу полів guardian_fields (ім'я та контакти опікуна для неповнолітніх студентів) та two_factor_secret для криптографічного захисту доступу.
2. Домен фінансового обліку (Billing & CRM Domain):
   - tariff_plans — довідник тарифних планів школи (назва пакету, вартість, кількість уроків у пакеті);
   - payments — таблиця транзакцій. Зберігає зв'язок між користувачем, обраним тарифом, сумою платежу, статусом транзакції (наприклад,

Approved або Declined від WayForPay) та кількістю зарахованих уроків.

3. Домен навчального процесу (LMS Domain):
   - courses — загальні навчальні напрямки (наприклад, "Основи Python", "QA Automation");
   - groups — конкретні навчальні потоки, що прив'язані до курсу та конкретного викладача;
   - enrollments — зв'язкова таблиця (Pivot Table) для реалізації відношення "Багато-до-багатьох" між студентами (users) та групами (groups);
   - lessons та lesson_contents — таблиці, що описують структуру конкретного заняття та зберігають прикріплені навчальні матеріали;
   - homework_submissions — таблиця для управління процесом оцінювання. Зберігає шляхи до завантажених файлів (assignment_file від викладача, student_file із розв'язком, teacher_file із фідбеком) та виставлену оцінку.
4. Домен планування (Scheduling Domain):
   - teacher_availabilities — матриця вільного часу. Зберігає дні тижня та часові проміжки, коли викладач готовий проводити заняття;
   - schedules — розклад занять. Зберігає прив'язку групи до часу, а також згенеровані через Zoom API посилання на відеокімнати (join_url, start_url). Важливим атрибутом є status (з можливістю збереження чернетки "draft").

База даних містить розгалужену систему зовнішніх ключів (Foreign Keys), які забезпечують підтримку посилальної цілісності на рівні рушія InnoDB. Основні типи зв'язків у системі:

- один-до-багатьох (1:N): наприклад, один користувач (Студент) може мати багато транзакцій у таблиці payments (user_id є зовнішнім ключем). Один

курс (Course) може мати багато груп (Group), а одна група — багато уроків (Lesson);

– багато-до-багатьох (M:N): студенти навчаються у багатьох групах, а група складається з багатьох студентів. Цей зв'язок на рівні СУБД розв'язано через проміжну таблицю enrollments, яка містить композитний ключ із user_id та group_id.

Для забезпечення оптимізації швидкості виконання SQL-запитів, на всі поля зовнішніх ключів та поля, за якими найчастіше відбувається пошук (наприклад, status у таблиці payments), були накладені B-Tree індекси.

У фреймворку Laravel фізичне створення таблиць та зв'язків реалізується не через написання сирих SQL-скриптів (DDL), а за допомогою версіонованих міграцій на мові PHP. Це дозволяє зберігати історію змін схеми бази даних у системі контролю версій (Git) та розгортати ідентичну структуру на різних серверах.

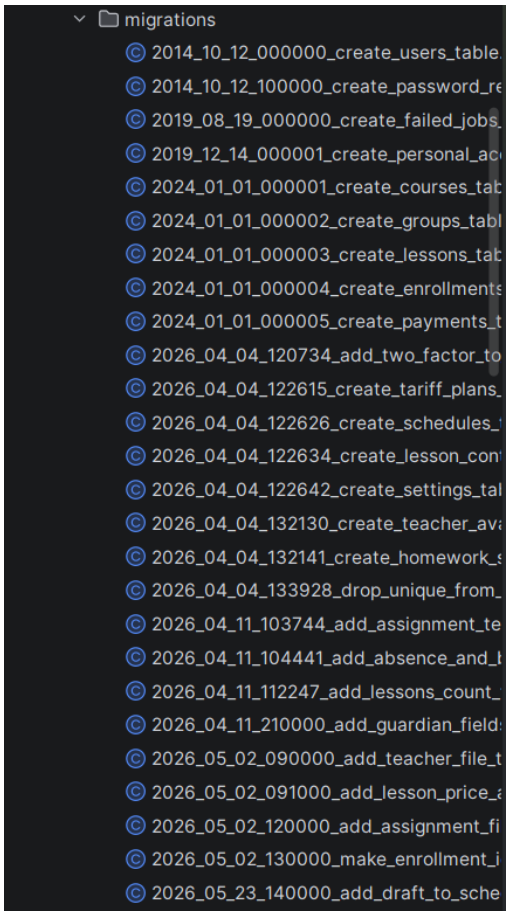


Рис. 2.5. Структура версіонованих міграцій бази даних розробленої системи

Нижче наведено лістинг коду міграції для створення комплексної таблиці homework_submissions, яка відіграє ключову роль у навчальному модулі. Вона демонструє використання зовнішніх ключів та полів для зберігання файлів:

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('homework_submissions', function (Blueprint
$table) {
            $table->id();
            $table->unsignedBigInteger('lesson_id');
            $table->unsignedBigInteger('student_id');
            $table->unsignedBigInteger('schedule_id')->nullable();
            $table->text('content')->nullable();
            $table->string('file_path')->nullable();
            $table->string('status')->default('pending'); //
pending, submitted, reviewed
            $table->string('grade')->nullable();
            $table->text('teacher_comment')->nullable();
            $table->timestamp('submitted_at')->nullable();
            $table->timestamp('reviewed_at')->nullable();
            $table->foreign('lesson_id')->references('id')-
>on('lessons')->cascadeOnDelete();
            $table->foreign('student_id')->references('id')-
>on('users')->cascadeOnDelete();
            $table->foreign('schedule_id')->references('id')-
>on('schedules')->nullOnDelete();
            $table->unique(['lesson_id', 'student_id']);
            $table->timestamps();
        });
    }
    public function down()
    {
        Schema::dropIfExists('homework_submissions');
    }
};
```

Для візуального відображення спроєктованої структури, усіх сутностей, їхніх атрибутів, первинних та зовнішніх ключів, а також кардинальності зв'язків

між ними, було побудовано комплексну ER-діаграму (діаграму "Сутність-Зв'язок"). Вона відображає концептуальну модель даних розробленої платформи.

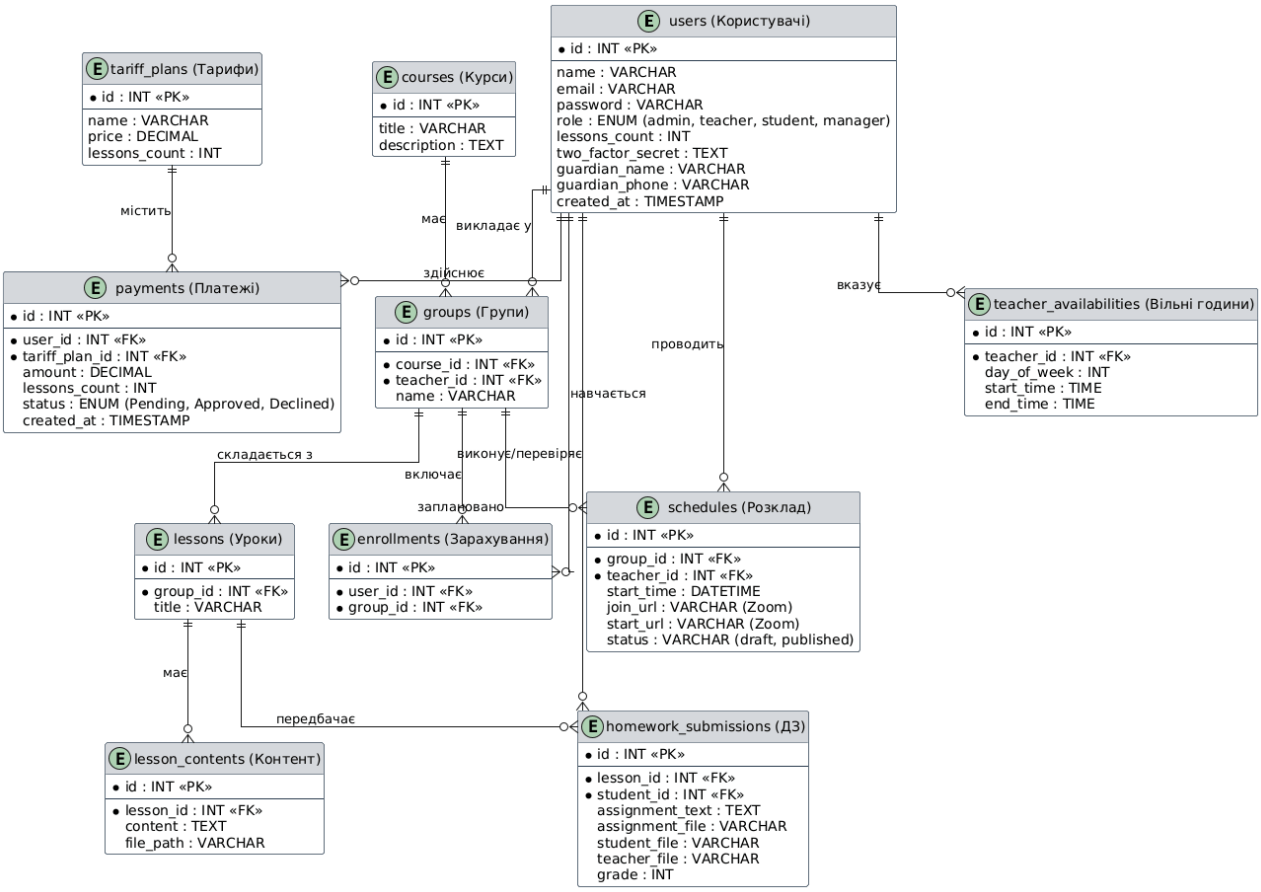


Рис. 2.6. ER-діаграма реляційної бази даних гібридної CRM/LMS системи

Спроєктована таким чином нормалізована структура реляційної бази даних гарантує надійне збереження фінансової, розкладової та академічної інформації, мінімізує дублювання даних та є міцним фундаментом для подальшої розробки клієнтських користувацьких інтерфейсів (Frontend).

2.4 Проєктування архітектури клієнтського застосунку та користувацьких інтерфейсів

Архітектура клієнтського застосунку (Frontend) відіграє вирішальну роль у забезпеченні зручної взаємодії користувача з розробленою гібридною CRM/LMS-системою. Враховуючи, що цільовою аудиторією платформи є адміністратори, викладачі та студенти з різним рівнем технічної підготовки, головними критеріями при проєктуванні інтерфейсу були обрані: інтуїтивна

зрозумілість, висока швидкість рендерингу сторінок (Performance), безпека передачі даних та повна адаптивність (Responsive Design) під мобільні пристрої.

На етапі проєктування було проведено аналіз двох основних архітектурних підходів до побудови клієнтської частини: SPA (Single Page Application на базі React/Vue) та SSR (Server-Side Rendering). Оскільки розроблювана система є закритою корпоративною платформою (де SEO-оптимізація публічних сторінок не є пріоритетом), але вимагає жорсткої зв'язності з бекендом та швидкої доставки контенту при першому завантаженні, було обрано підхід серверного рендерингу з використанням вбудованого у фреймворк Laravel шаблонізатора Blade.

Шаблонізатор Blade компілює представлення (Views) у чистий PHP-код і кешує їх на рівні файлової системи сервера, що зводить накладні витрати на рендеринг майже до нуля. Для забезпечення принципу DRY (Don't Repeat Yourself) та полегшення подальшої підтримки коду, архітектура інтерфейсу побудована на основі ієрархічного успадкування та компонентного підходу.

Усі сторінки системи успадковують базовий каркас (Layout) із файлу resources/views/layouts/app.blade.php. Складні або повторювані елементи інтерфейсу інкапсульовані в окремі анонімні компоненти. Наприклад, для уніфікації заголовків на всіх сторінках системи (курси, розклад, фінанси) було створено компонент <x-page-header>.

Нижче наведено лістинг коду розробленого компонента resources/views/components/page-header.blade.php, який приймає параметри (props) через атрибути:

```
<div class="flex items-center justify-between mb-6">
    <div>
        <h2 class="text-lg font-semibold text-slate-800">{{ $title
}}</h2>
        @isset($subtitle)<p class="text-sm text-slate-500 mt-
0.5">{{ $subtitle }}</p>@endisset
    </div>
    @isset($action)
    <a href="{{ $action['url'] }}"
       class="inline-flex items-center gap-2 bg-indigo-500
hover:bg-indigo-600 text-white text-sm font-medium px-4 py-2
rounded-xl transition shadow-sm shadow-indigo-200">
```

```
        <i class="fa fa-plus text-xs"></i> {{ $action['label'] }}
    </a>
    @endisset
</div>
```

Виклик цього компонента на сторінці списку курсів виглядає максимально лаконічно: <x-page-header title="Управління курсами" buttonText="Додати курс" buttonRoute="courses.create" />, що значно зменшує дублювання HTML-розмітки.

Особливу увагу при проєктуванні користувацького інтерфейсу було приділено безпеці передачі даних та обробці користувацького вводу. Усі HTML-форми в системі (авторизація, створення розкладу, завантаження домашніх завдань) обов'язково захищені від атак підробки міжсайтових запитів (CSRF) за допомогою Blade-директиви @csrf, яка генерує прихований input з унікальним криптографічним токеном.

Для забезпечення якісного користувацького досвіду (UX), логіка бекенд-валідації тісно інтегрована з фронтендом. Якщо дані не проходять перевірку в контролері (наприклад, користувач ввів занадто короткий пароль або не завантажив файл), система автоматично перенаправляє його назад із збереженням попередньо введених даних (функція old()) та передачею масиву помилок.

Приклад реалізації безпечної форми авторизації (resources/views/auth/login.blade.php) з відображенням помилок валідації:

```
<form method="POST" action="{{ route('login') }}" class="space-y-
5">
    @csrf
    <div>
        <label class="block text-sm font-medium text-slate-700 mb-
1.5">Email</label>
        <div class="relative">
            <span class="absolute inset-y-0 left-3 flex items-
center text-slate-400">
                <i class="fa fa-envelope text-sm"></i>
            </span>
            <input type="email" name="email" value="{{
old('email') }}" required autofocus
                   class="w-full pl-10 pr-4 py-2.5 border border-
slate-300 rounded-xl text-sm focus:outline-none focus:ring-2
focus:ring-indigo-500 focus:border-transparent transition"
                   placeholder="admin@school.com">
```

```
        </div>
    </div>
    <button type="submit"
            class="w-full bg-indigo-500 hover:bg-indigo-600 text-
white font-semibold py-2.5 rounded-xl text-sm transition shadow-lg
shadow-indigo-500/25">
        Увійти
    </button>
</form>
```

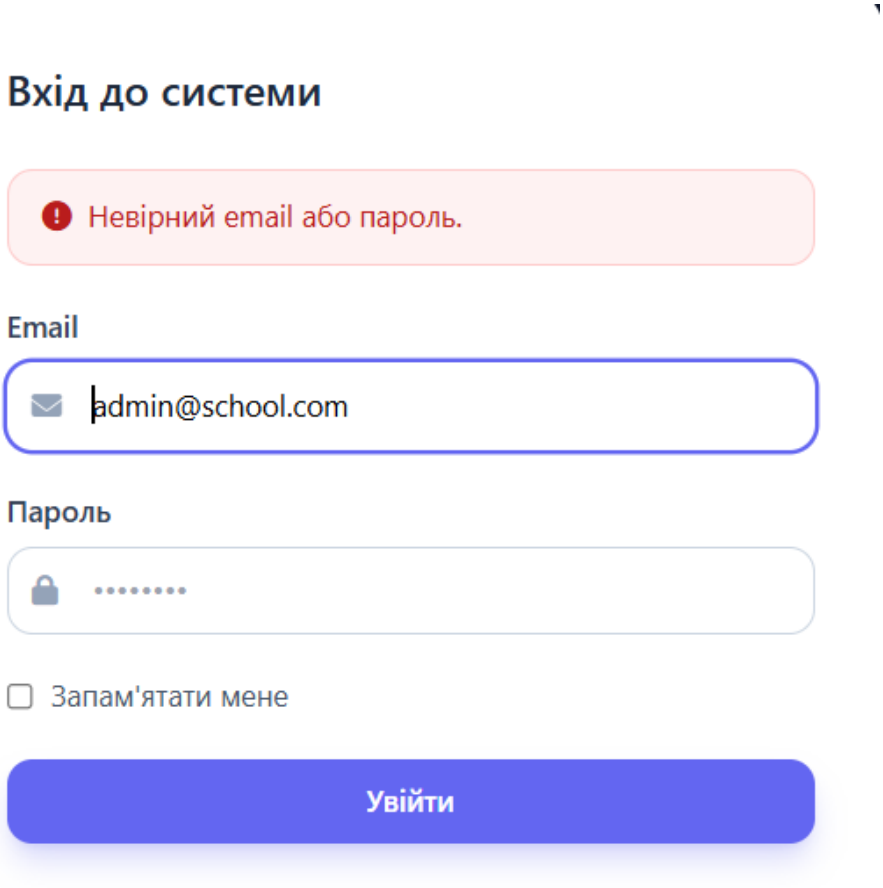


Рис. 2.7. Інтерфейс авторизації користувача з інтегрованою системою виведення помилок валідації

Для візуального оформлення інтерфейсу було відкинуто традиційний семантичний підхід та обрано утилітарний фреймворк Tailwind CSS. Цей інструмент дозволяє стилізувати компоненти безпосередньо в HTML-розмітці за допомогою низькорівневих класів.

Враховуючи вимогу до максимальної простоти розгортання проєкту на сервері та відмову від складних Node.js-залежностей, стилізацію було реалізовано за допомогою підходу Tailwind Play CDN. У базовому шаблоні app.blade.php фреймворк підключається безпосередньо через тег <script>.

Локальна конфігурація теми (кастомні кольори primary та sidebar) ініціалізується безпосередньо в об'єкті tailwind.config у секції <head>:

```
<script src="https://cdn.tailwindcss.com"></script>
<script>
    tailwind.config = {
        theme: {
            extend: {
                colors: {
                    primary: { DEFAULT: '#6366f1', light:
'#818cf8', dark: '#4f46e5' },
                    sidebar: '#0f172a',
                }
            }
        }
    }
</script>
```

Такий підхід дозволив миттєво застосовувати зміни стилів без необхідності використання локальних збирачів ресурсів (Webpack чи Vite) та забезпечив високу гнучкість розробки. Додаткові клієнтські скрипти та стилі підключаються динамічно лише на тих сторінках, де вони потрібні, за допомогою Blade-директив @stack('styles') та @stack('scripts').

Для проєктування оптимального користувацького досвіду (UX) та забезпечення безперешкодного переміщення користувача між ключовими модулями було побудовано карту станів навігації (Navigation Map).

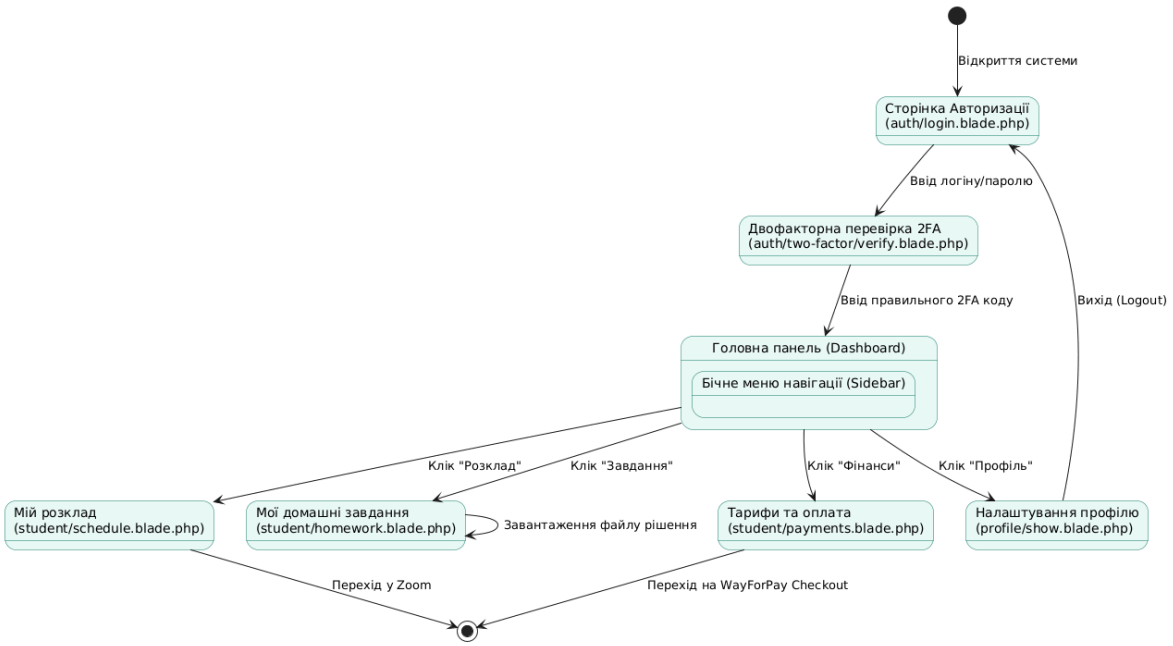


Рис. 2.8. Схема навігації (UI/UX) та граф переходів між сторінками особистого кабінету студента

Завдяки синергії технологій Blade (для SSR рендерингу та компонентизації), Tailwind CSS (для швидкодії стилів) та Vite (для збирання скриптів), розроблена архітектура клієнтської частини забезпечує надійне, безпечне та візуально привабливе відображення даних, що повністю задовольняє вимоги, поставлені перед CRM/LMS-системою IT-школи.

Висновки до другого розділу

У другому розділі кваліфікаційної роботи було виконано детальне проєктування архітектури гібридної CRM/LMS-системи, що охопило основні етапи створення програмного продукту: від аналізу функціональних вимог до побудови структури бази даних і клієнтського інтерфейсу.

На етапі моделювання було сформовано 15 основних функціональних прецедентів (Use Cases), які визначили ключові сценарії взаємодії в системі та розмежували права доступу між чотирма ролями: адміністратором, менеджером, викладачем і студентом. Це дозволило чітко визначити функціональні межі кожного користувача та логіку роботи системи.

Для реалізації серверної частини за основу було обрано архітектуру MVC, яку доповнено патернами Service Layer та Repository. Такий підхід дозволив підвищити гнучкість системи, спростити її підтримку та забезпечити кращу ізоляцію бізнес-логіки. Особливо це важливо для інтеграції зовнішніх сервісів, зокрема платіжної системи WayForPay та API сервісу Zoom.

Окрему увагу було приділено проєктуванню реляційної бази даних. Її структура була нормалізована до третьої нормальної форми (3NF), що забезпечило мінімізацію дублювання даних та підтримку цілісності зв’язків між сутностями. Реалізація бази даних у MySQL за допомогою міграцій Laravel дозволила створити надійну основу для роботи з фінансами, розкладом занять, групами та домашніми завданнями.

Також було спроєктовано клієнтську частину системи із використанням серверного рендерингу (SSR), шаблонізатора Blade, Tailwind CSS та Vite. Це дозволило сформувати продуктивний, безпечний і зручний інтерфейс для всіх категорій користувачів.

РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ CRM-СИСТЕМИ

3.1 Практична реалізація модулів серверної частини та інтеграція зовнішніх сервісів

Етап програмної реалізації передбачає перенесення спроєктованої багаторівневої архітектури (Service-Repository Pattern) у площину реального програмного коду. Усі компоненти серверної частини розроблені на мові PHP з суворим дотриманням міжнародних стандартів кодування PSR-12, що забезпечує високу читабельність та легкість подальшого рефакторингу [20]. Зважаючи на багатовекторність розробленої CRM/LMS-системи, процес програмування був декомпозований на три масштабні підсистеми: маршрутизація та безпека, фінансовий білінг, а також автоматизація академічних процесів (інтеграції та LMS).

Забезпечення захисту корпоративних та персональних даних реалізовано на кількох рівнях архітектури Laravel. Першим рівнем захисту є строга типізація та санація всіх вхідних даних від клієнта. Перевірка даних імплементована безпосередньо на рівні HTTP-координаторів (контролерів) за допомогою вбудованого методу validate(). Цей підхід гарантує, що до шару бізнес-логіки (Services) потрапляють виключно очищені дані, що унеможливлює XSS або SQL-ін'єкції [21].

Важливою архітектурною особливістю розробленого коду є використання сучасних можливостей PHP 8, зокрема Constructor Property Promotion для впровадження залежностей (Dependency Injection). Це чітко простежується на прикладі реалізованого CourseController:

```
namespace App\Http\Controllers;

use App\Models\Course;
use App\Services\CourseService;
use App\Services\UserService;
use Illuminate\Http\Request;

class CourseController extends Controller
{
    public function __construct(
```

```
        protected CourseService $courseService,
        protected UserService $userService
    ) {}
    public function store(Request $request)
    {
    $data = $request->validate([
        'title'          => ['required', 'string', 'max:255'],
        'description'    => ['nullable', 'string'],
        'price'          => ['required', 'numeric', 'min:0'],
        'lesson_price'   => ['nullable', 'numeric', 'min:0'],
        'duration_hours' => ['required', 'integer', 'min:1'],
        'level'          => ['required',
'in:beginner,intermediate,advanced'],
        'status'         => ['required',
'in:active,inactive,archived'],
        'teacher_id'     => ['nullable', 'exists:users,id'],
        'image_file'     => ['nullable', 'image', 'max:2048'],
    ]);

    $this->courseService->create($data);
    return redirect()->route('courses.index')->with('success',
'Курс створено успішно.');
}
}
```

Другим рівнем є управління доступом (Role-Based Access Control). У системі реалізовано розмежування прав доступу на основі ідентифікатора ролі (role_id), де 1 відповідає «Адміністратору», а 2 — «Менеджеру». Додатково в моделі користувача реалізовані методи-хелпери (наприклад, isTeacher(), isStudent()), які використовуються як для захисту маршрутів на бекенді, так і для динамічного рендерингу елементів інтерфейсу у Blade-шаблонах:

```
@if(auth()->user()->role_id <= 2)
    <a href="{{ route('dashboard') }}">Дашборд</a>
    <a href="{{ route('courses.index') }}">Курси</a>
@endif

@if(auth()->user()->isTeacher())
    <a href="{{ route('teacher.lessons') }}">Мої курси та
уроки</a>
@endif
```

Для систематизації прав доступу було розроблено матрицю авторизації (табл. 3.1).

Таблиця 3.1

Матриця розподілу прав доступу (RBAC) до модулів CRM-системи

Модуль (Маршрут / Route)	**Адміністратор**	**Менеджер**	**Викладач**	**Студент**
settings.* (Налаштування API Zoom, WayForPay)	**Доступно**	Заборонено	Заборонено	Заборонено
courses.*, groups.* (Управління програмами)	**Доступно**	**Доступно** (Read Only)	Заборонено	Заборонено
payments.* (Моніторинг транзакцій клієнтів)	**Доступно**	**Доступно**	Заборонено	Заборонено
teacher.homework.* (Оцінювання завдань)	Заборонено	Заборонено	**Доступно**	Заборонено
student.schedule.*, student.payments.*	Заборонено	Заборонено	Заборонено	**Доступно**

Третім рівнем безпеки є реалізація алгоритму Time-Based One-Time Password (TOTP) за допомогою інтеграції пакета pragmarx/google2fa-laravel. У базі даних передбачено поле two_factor_secret. Якщо воно заповнене, TwoFactorMiddleware тимчасово обмежує доступ до системи після вводу пароля та перенаправляє користувача на контролер для перевірки 6-значного коду. Така імплементація відповідає міжнародному стандарту RFC 6238 [22].

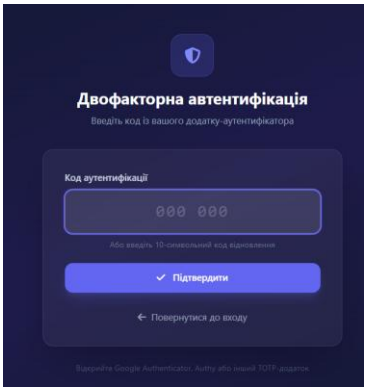


Рис. 3.1. Програмна реалізація інтерфейсу двофакторної автентифікації (2FA)

Модуль фінансового білінгу є критичним вузлом інфраструктури, який відповідає за продаж тарифних планів та облік занять. Згідно з принципом єдиної відповідальності (SRP), логіку взаємодії з платіжним шлюзом (WayForPay) та логіку роботи з балансами студентів відокремлено від основних контролерів у відповідні сервісні класи [23].

Процес обробки фінансів складається з ініціалізації платежу (з обов'язковою генерацією підпису HMAC_MD5 на сервері для уникнення підміни суми клієнтом) та обробки асинхронного Webhook-запиту від банку.

Найскладнішим інженерним завданням при розробці фінансової частини було уникнення стану гонитви (Race Condition). Оскільки викладач може завершувати урок, а клієнт одночасно оплачувати новий пакет, оновлення бази даних здійснюється в ізольованому середовищі. Застосування песимістичного блокування (lockForUpdate()) на рівні транзакцій рушія InnoDB гарантує, що рядок користувача блокується для інших процесів до моменту завершення математичних операцій з його балансом (lessons_count) [24].

Процес формування віртуальних кімнат для занять автоматизовано за допомогою інтеграції із Zoom API. Авторизація бекенду відбувається за сучасним протоколом Server-to-Server OAuth 2.0 [25]. Система надсилає HTTP-запит до серверів Zoom, отримує тимчасовий Bearer Token і формує параметри зустрічі, зберігаючи отримані посилання (join_url та start_url) у базу даних.

Для наочної візуалізації логіки взаємодії об'єктів розроблено UML-діаграму комунікації (рис. 3.2).

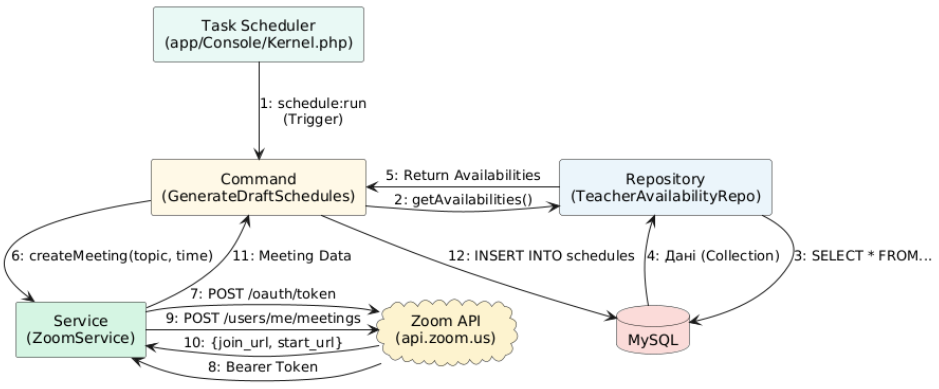


Рис. 3.2. UML-діаграма комунікації підсистеми генерації розкладів та взаємодії із зовнішнім API

Навчальний (LMS) модуль забезпечує обробку та перевірку домашніх завдань. Для автоматизації роботи викладача була реалізована складна програмна логіка підрахунку завдань, що очікують на перевірку. Замість неефективних циклів на рівні PHP, у системі розроблено оптимізований Eloquent-запит, який безпосередньо в базовому шаблоні (app.blade.php) обчислює кількість таких завдань для виведення бейджа-сповіщення:

```
@php $pendingHw =
\App\Models\HomeworkSubmission::whereIn('lesson_id',
    \App\Models\Lesson::whereIn('course_id',
        \App\Models\Course::where('teacher_id', auth()->id())
            ->orWhereHas('groups', fn($q) => $q-
>where('teacher_id', auth()->id()))
            ->pluck('id')
    )->pluck('id')
)->where('status','submitted')->count(); @endphp
<a href="{{ route('teacher.homework.index') }}"
   class="sidebar-item flex items-center gap-3 px-3 py-2.5
rounded-lg text-sm hover:bg-slate-800 transition {{ request()-
>routeIs('teacher.homework*') ? 'active text-indigo-400' : 'text-
slate-300' }}">
    <i class="fa fa-pen-to-square w-4 text-center"></i>
    <span>Домашні завдання</span>
    @if($pendingHw > 0)
    <span class="ml-auto bg-amber-500 text-white text-xs font-bold
px-1.5 py-0.5 rounded-full flex-shrink-0">{{ $pendingHw }}</span>
    @endif
</a>
```

Наведений фрагмент програмного коду ілюструє застосування розширених можливостей об’єктно-реляційного відображення (ORM) Eloquent для вирішення складних завдань багаторівневої вибірки пов'язаних даних. Замість того, щоб завантажувати у пам’ять сервера масиви об’єктів (курсів, уроків та виконаних завдань) для їх подальшої фільтрації циклами засобами мови PHP, уся логіка підрахунку ефективно делегована рушію бази даних. Це дозволяє уникнути класичної проблеми зниження продуктивності «N+1 запитів» та критичного переповнення оперативної пам'яті (Memory Leak) при збільшенні кількості користувачів.

Загалом, даний програмний блок підтверджує можливість ефективного використання абстракцій фреймворку для роботи з глибоко вкладеними реляційними зв'язками, забезпечуючи при цьому високу швидкість

відмальовування інтерфейсу сторінки (Page Load Time). Для алгоритмізації дій викладача та студента у процесі перевірки завдання було розроблено діаграму діяльності (рис. 3.3).

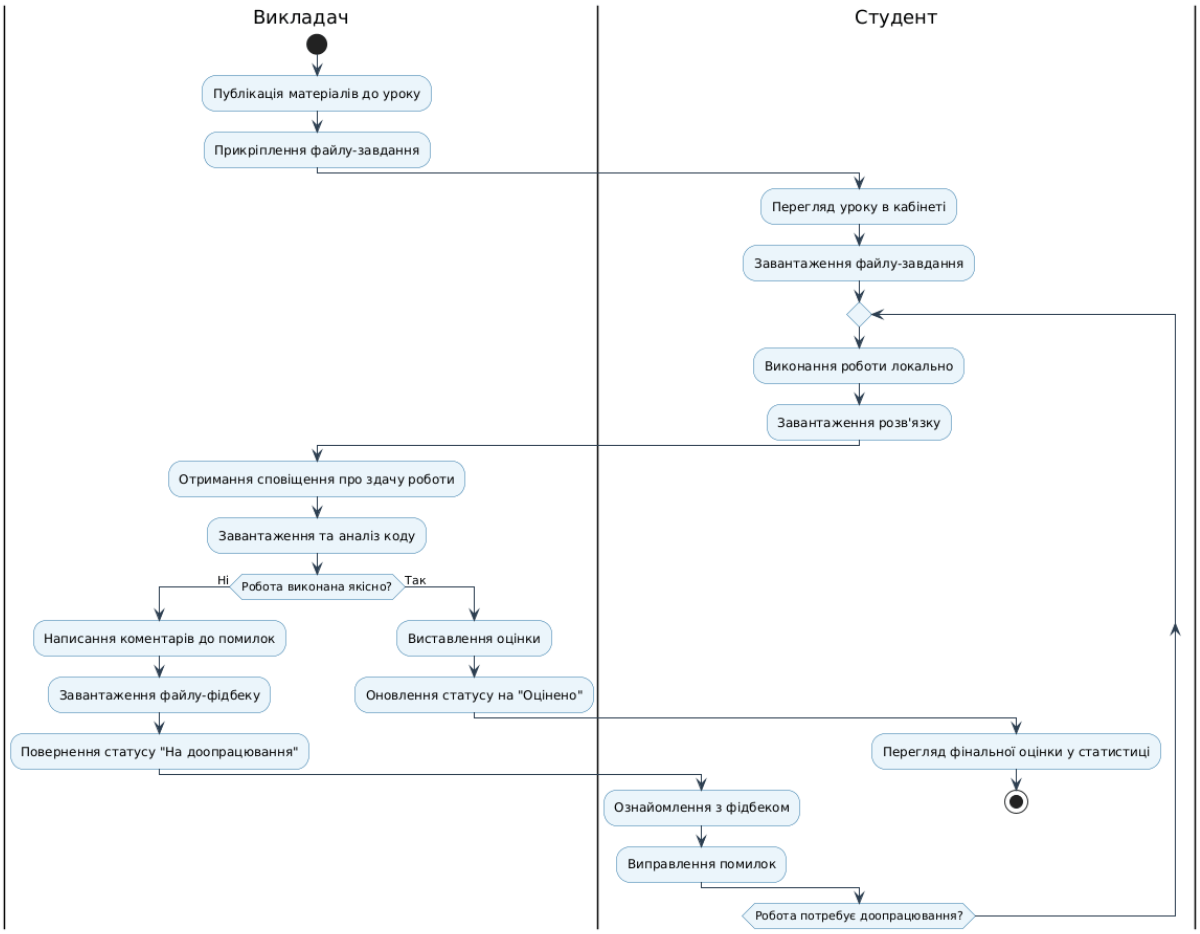


Рис. 3.3. UML-діаграма діяльності (Activity) процесу виконання та оцінювання домашнього завдання

Налагоджена взаємодія між розробленими модулями доводить ефективність застосування обраних архітектурних рішень, що гарантують безпеку даних та високу масштабованість бекенд-інфраструктури.

3.2 Розробка клієнтського застосунку та опис кабінетів користувачів

Успішна експлуатація розробленої гібридної CRM/LMS-системи безпосередньо залежить від якості клієнтського застосунку (Frontend). Оскільки цільовою аудиторією платформи є користувачі з різним рівнем технічної підготовки (від системних адміністраторів до студентів та їхніх опікунів),

інтерфейс був спроєктований з акцентом на інтуїтивну зрозумілість, мінімалізм та високу швидкість відмальовування.

Для реалізації клієнтської частини було використано технологію серверного рендерингу (Server-Side Rendering) на базі шаблонізатора Blade, що вбудований у фреймворк Laravel [26]. Для візуального оформлення (стилізації) використано утилітарний фреймворк Tailwind CSS, який підключається через мережу доставки контенту (CDN). Це дозволило відмовитися від складних локальних збирачів ресурсів та пришвидшити процес розгортання проєкту [27].

Архітектура інтерфейсу побудована за компонентним принципом з використанням єдиного базового шаблону — resources/views/layouts/app.blade.php. Цей файл містить спільні для всіх сторінок елементи: підключення шрифтів, бібліотеки іконок FontAwesome, навігаційну панель (Sidebar), верхній колонтитул (Header) та область для динамічного контенту (@yield('content')).

Важливою складовою розробки стало забезпечення адаптивності (Responsive Web Design) [28]. У базовому шаблоні реалізовано мобільне меню (Sidebar Overlay), яке керується за допомогою нативного JavaScript без використання важких бібліотек типу jQuery:

```
<div class="flex-1 flex flex-col overflow-hidden min-w-0">
    <header class="bg-white border-b border-slate-200 px-4 lg:px-6
py-4 flex items-center justify-between flex-shrink-0">
        <div class="flex items-center gap-3">
            <button onclick="openSidebar()" class="lg:hidden text-
slate-500 hover:text-slate-800 transition p-1">
                <i class="fa fa-bars text-lg"></i>
            </button>
            <h1 class="text-lg lg:text-xl font-semibold text-
slate-800">@yield('header', 'Дашборд')</h1>
        </div>
        <div class="flex items-center gap-2 lg:gap-4">
            <span class="hidden sm:block text-sm text-slate-
400">{{ now()->isoFormat('D MMMM YYYY') }}</span>
            <form method="POST" action="{{ route('logout') }}">
                @csrf
                <button type="submit"
                        class="inline-flex items-center gap-2
text-sm text-slate-500 hover:text-red-600 hover:bg-red-50 border
border-slate-200 hover:border-red-200 px-3 py-1.5 rounded-xl
transition">
```

```
                    <i class="fa fa-sign-out-alt text-
xs"></i><span class="hidden sm:inline">Вийти</span>
                </button>
            </form>
        </div>
    </header>
<script>
function openSidebar() {
    document.getElementById('sidebar').classList.remove('-
translate-x-full');
    document.getElementById('sidebar-
overlay').classList.remove('hidden');
}
function closeSidebar() {
    document.getElementById('sidebar').classList.add('-translate-
x-full');
    document.getElementById('sidebar-
overlay').classList.add('hidden');
}
</script>
```

Щоб не створювати чотири окремі шаблони для кожної ролі, у системі реалізовано динамічний рендеринг навігаційного меню на основі поточного контексту авторизації. Використовуючи Blade-директиви @if та власні методи моделі користувача (isTeacher(), isStudent()), система генерує лише ті пункти меню, до яких користувач має доступ згідно з рольовою моделлю.

Таблиця 3.2 деталізує розподіл функціональних модулів користувацького інтерфейсу (UI) залежно від типу особистого кабінету.

Таблиця 3.2

Структура особистих кабінетів користувачів CRM-системи

Назва модуля (Пункт меню)	**Кабінет Адміністратора**	**Кабінет Викладача**	**Кабінет Студента**
Головна панель (Дашборд)	Аналітика, графіки оплат	Відсутній	Відсутній
Курси та Уроки	Повне CRUD-управління	Лише перегляд своїх курсів	Відсутній
Розклад занять	Загальний розклад школи	Особистий розклад викладача	Особистий розклад студента
Домашні завдання	Відсутній	Перевірка та оцінювання	Завантаження розв'язків
Фінанси та Оплата	Керування транзакціями	Відсутній	Історія оплат та поповнення
Налаштування системи	Zoom API, WayForPay, Тарифи	Відсутній	Відсутній

Для візуалізації процесу генерації користувацького інтерфейсу на стороні сервера було побудовано UML-діаграму станів (рис. 3.4).

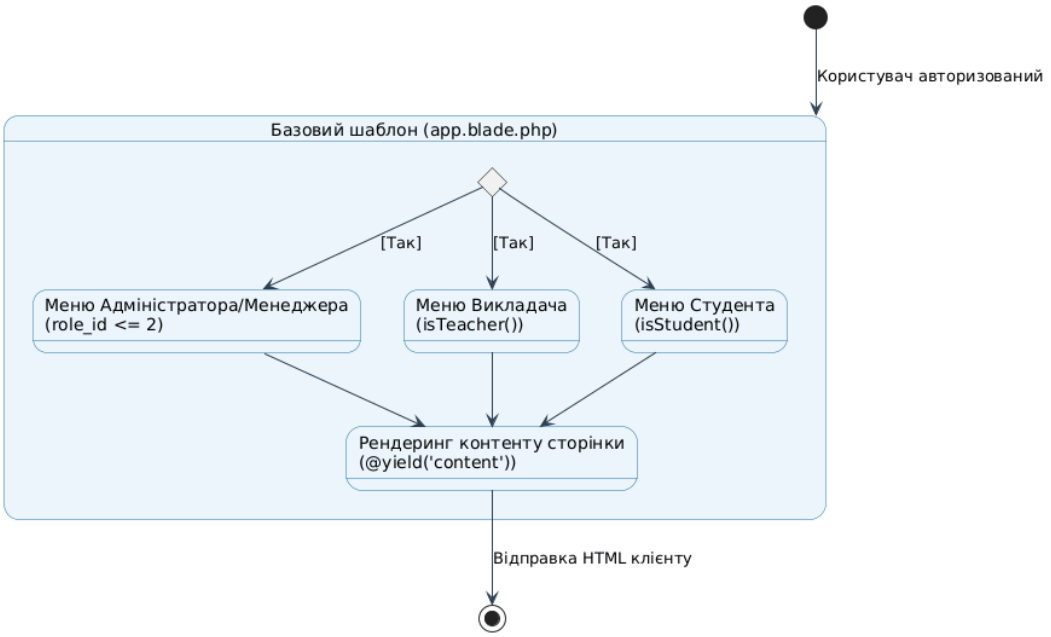


Рис. 3.4. UML-діаграма станів генерації динамічного навігаційного меню (Sidebar)

Кабінет адміністративного персоналу орієнтований на управління бізнес-процесами IT-школи. Згідно з кодом шаблону app.blade.php, блок адміністратора (auth()->user()->role_id <= 2) містить найбільшу кількість інструментів.

До ключових інтерфейсів цього кабінету належать:

- управління групами та студентами (groups.index, enrollments.index): інтерфейс представляє собою таблиці даних із можливістю пошуку та фільтрації. У цих розділах менеджери закріплюють студентів за певними навчальними потоками;
- моніторинг платежів (payments.index): сторінка з фінансовою аналітикою, де відображаються статуси транзакцій від WayForPay;
- глобальні налаштування (тільки role_id === 1): спеціальний розділ для системного адміністратора, де конфігуруються тарифи (tariff-plans.index) та оновлюються ключі доступу до зовнішніх API.

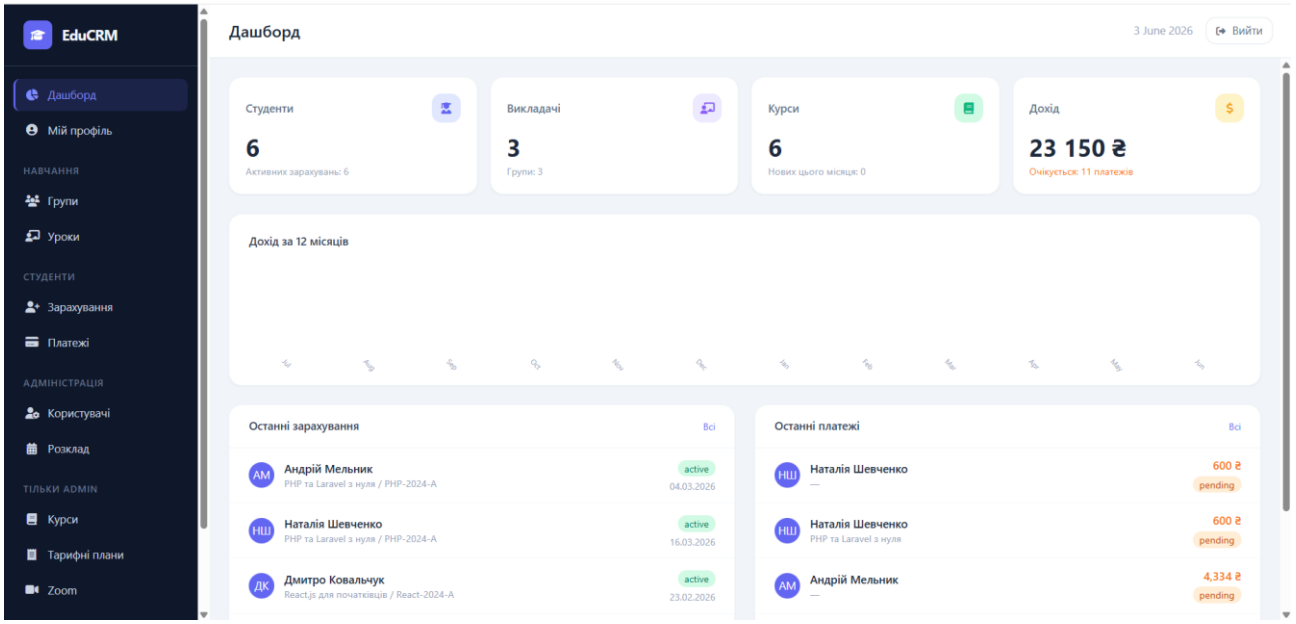


Рис. 3.5. Інтерфейс особистого кабінету «Адміністратора»

Інтерфейс викладача спроєктований таким чином, щоб мінімізувати відволікання від навчального процесу. Блок меню викладача (auth()->user()->isTeacher()) містить доступ до матриці вільного часу (teacher.availability), де викладач за допомогою зручної форми обирає години, коли він готовий проводити заняття.

Особливістю клієнтського застосунку викладача є динамічний лічильник домашніх завдань, що потребують перевірки. Програмна логіка Blade-шаблону підраховує кількість робіт у статусі submitted та виводить яскравий бейдж безпосередньо в бічному меню:

```
<a href="{{ route('teacher.homework.index') }}"
   class="sidebar-item flex items-center gap-3 px-3 py-2.5
rounded-lg text-sm hover:bg-slate-800 transition {{ request()-
>routeIs('teacher.homework*') ? 'active text-indigo-400' : 'text-
slate-300' }}">
    <i class="fa fa-pen-to-square w-4 text-center"></i>
    <span>Домашні завдання</span>
    @if($pendingHw > 0)
    <span class="ml-auto bg-amber-500 text-white text-xs font-bold
px-1.5 py-0.5 rounded-full flex-shrink-0">{{ $pendingHw }}</span>
    @endif
</a>
```

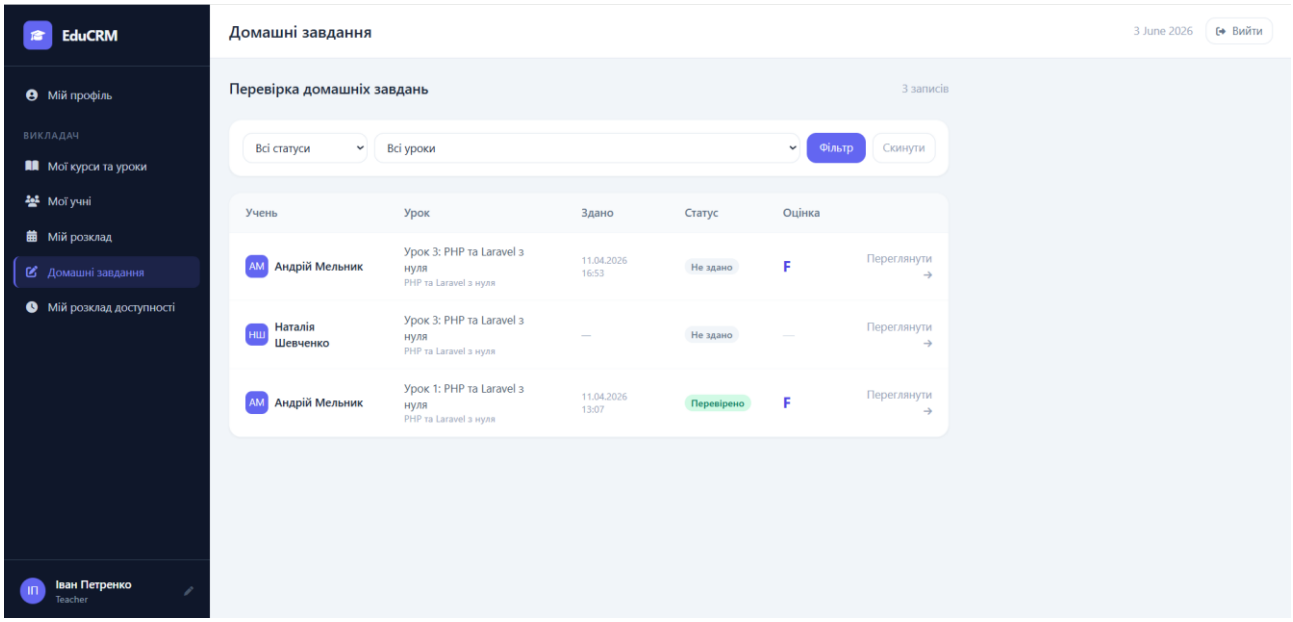


Рис. 3.6. Інтерфейс кабінету «Викладача» та модуль перевірки домашніх завдань

Кабінет студента є основним продуктом, з яким взаємодіє кінцевий споживач. Меню студента (auth()->user()->isStudent()) зведено до чотирьох базових сторінок:

– мій розклад (student.schedule): сторінка відображає хронологічний список майбутніх занять. Головним елементом взаємодії є кнопка "Долучитися", яка містить згенероване через Zoom API посилання join_url;
– домашні завдання (student.homework): інтерфейс завантаження архівів із виконаними роботами та перегляду фідбеку (оцінки) від викладача;
– оплата (student.payments): інтерфейс, у якому студент або його опікун можуть переглянути поточний баланс занять (поле lessons_count), обрати тарифний план і перейти на сторінку еквайрингу WayForPay для поповнення рахунку.

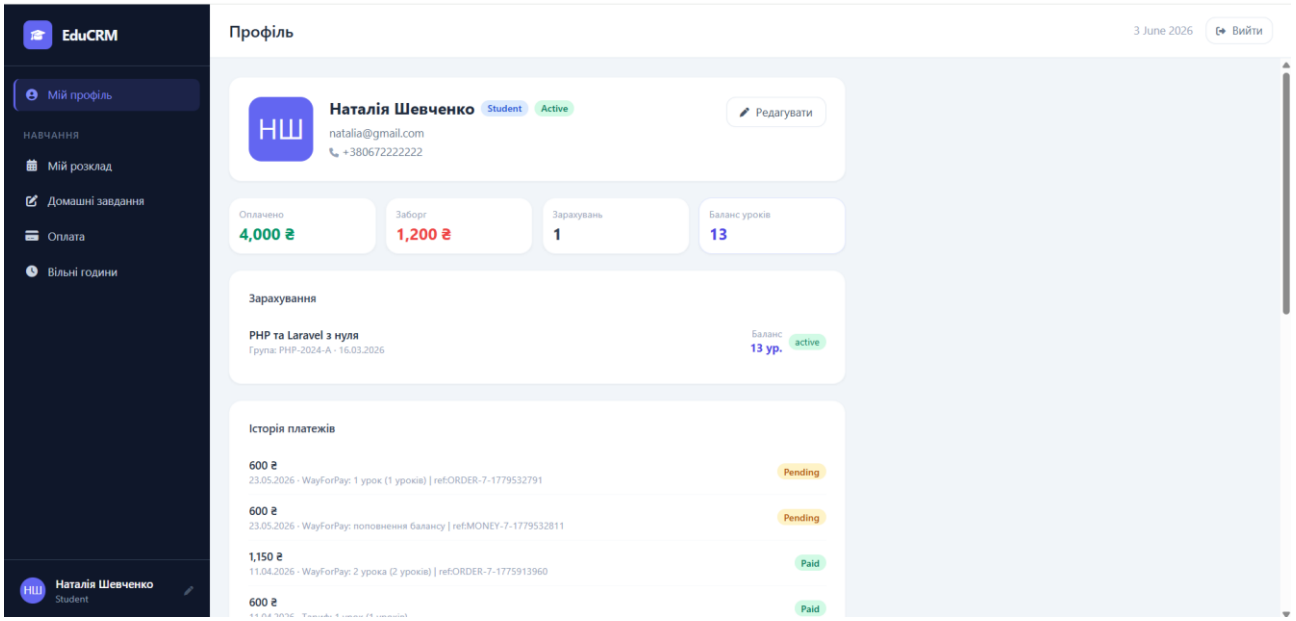


Рис. 3.7. Інтерфейс особистого кабінету «Студента»

Завдяки застосуванню Blade-компонентів, CDN-інтеграції Tailwind CSS та продуманій структурі базового макета (Layout), розроблений клієнтський застосунок працює швидко, повністю відповідає вимогам адаптивності та забезпечує високий рівень користувацького досвіду (UX) для всіх ролей системи.

3.3 Опис роботи з системою

Проєктування інтерфейсу користувача (UI) та користувацького досвіду (UX) є критично важливим етапом розробки, оскільки цільова аудиторія системи охоплює користувачів з різним рівнем технічної підготовки. Гібридна CRM/LMS-система побудована на базі адаптивного вебдизайну (Tailwind CSS), що забезпечує коректне відображення як на персональних комп'ютерах, так і на мобільних пристроях.

Нижче наведено детальний опис життєвого циклу взаємодії користувачів із системою, поділений за функціональними ролями. Робота з платформою розпочинається зі сторінки авторизації (/login). Користувач вводить свою електронну пошту та пароль. Якщо для облікового запису активовано двофакторну автентифікацію (2FA), система тимчасово призупиняє сесію і перенаправляє користувача на сторінку auth.two-factor.verify. Для продовження

роботи необхідно ввести 6-значний код із мобільного застосунку (наприклад, Google Authenticator).

Після успішної перевірки ідентичності в дію вступає програмна логіка DashboardController. Система автоматично визначає роль користувача і здійснює перенаправлення (Redirect) до відповідного робочого простору:

– викладачі автоматично перенаправляються до списку своїх учнів та груп;
– студенти спрямовуються на сторінку свого особистого розкладу занять;
– адміністратори та менеджери потрапляють на головну аналітичну панель (Dashboard).

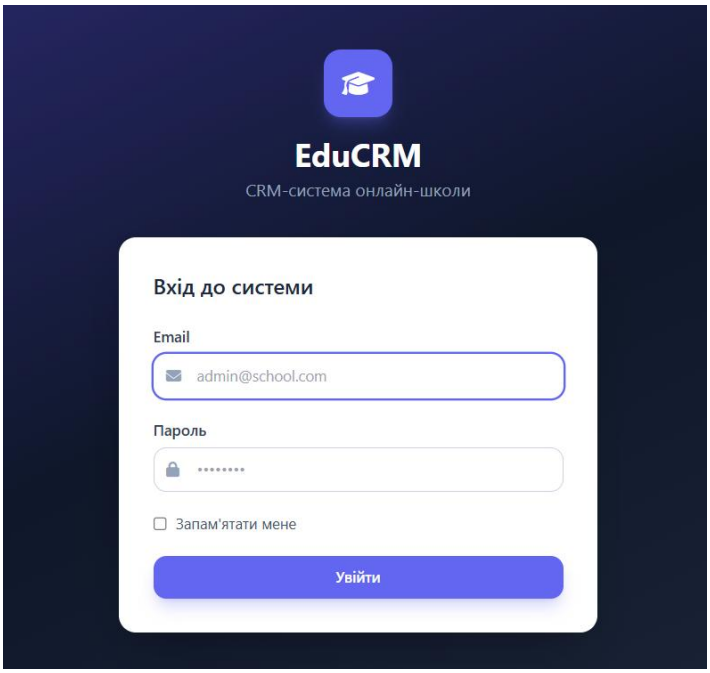


Рис. 3.8. Інтерфейс авторизації

Кабінет «Адміністратора» та «Менеджера» орієнтований на управління бізнес-процесами школи. Головна панель (dashboard.index) надає зведену аналітичну інформацію щодо загального стану системи.

Навігаційне меню (Сайдбар) забезпечує швидкий доступ до ключових модулів керування:

– управління контингентом («Користувачі», «Групи», «Записи на курси»): менеджер має змогу створювати профілі нових студентів, формувати навчальні групи (groups.index) та закріплювати студентів за конкретними курсами через модуль Enrollments. Для кожного студента ведеться облік

його балансу занять (lessons_balance) та грошового балансу (money_balance);

– фінансовий моніторинг (Платежі): модуль відображає історію всіх транзакцій у системі. Менеджер може відстежувати статуси платежів (pending, paid) та, за потреби, вручну додавати платежі готівкою, вказуючи відповідні примітки;

– налаштування та тарифи (Доступно лише «Адміністраторам»): системний адміністратор має доступ до конфігурації тарифних планів (tariff-plans.index), де вказується кількість уроків та їхня вартість. Також у розділі «Налаштування» здійснюється управління API-ключами платіжного шлюзу WayForPay та платформи Zoom.

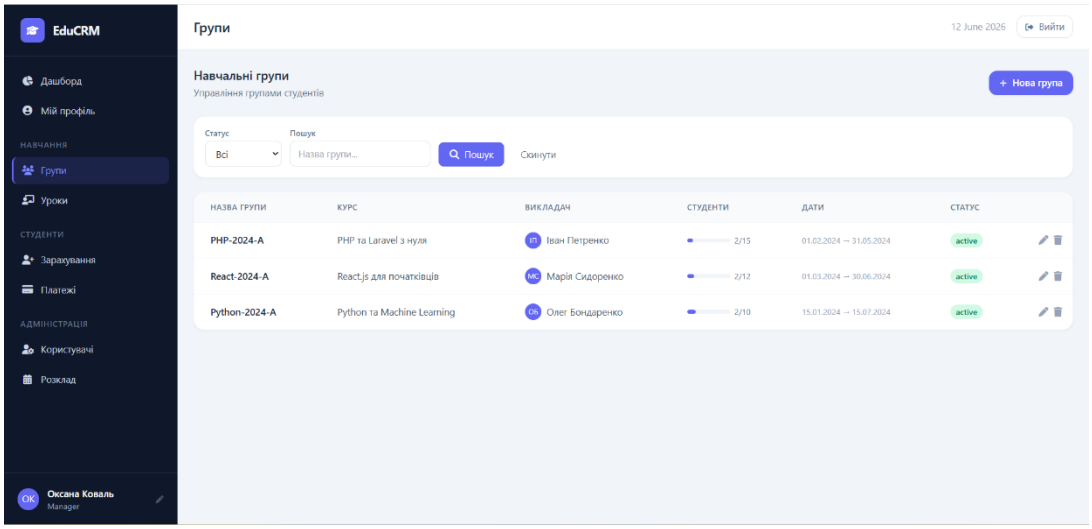


Рис. 3.9. Робочий простір «Менеджера»: управління групами

Інтерфейс викладача спроєктовано з акцентом на мінімізацію відволікаючих факторів. Головним завданням викладача є проведення занять та перевірка успішності.

Ключові функціональні можливості кабінету:

– матриця доступності (teacher.availability): викладач самостійно визначає свій робочий графік. У спеціальній формі він обирає дні тижня та часові слоти (start_time, end_time), коли він готовий проводити заняття. Ці дані використовуються консольною командою генерації розкладів для математичного пошуку перетинів із вільним часом студентів;

– перегляд розкладу та уроків: викладач має доступ до списку своїх груп (teacher.students) та розкладу майбутніх сесій із посиланнями для початку конференції (Zoom Start URL);
– модуль домашніх завдань (teacher.homework.index): найбільш інтерактивний розділ. Викладач бачить список робіт, які очікують на перевірку (статус submitted). При відкритті завдання викладач може завантажити файл студента, ознайомитися з ним, після чого через вебформу виставити оцінку (grade), написати текстовий коментар (teacher_comment) та прикріпити файл зі зворотним зв'язком. Після збереження статус роботи змінюється на reviewed.

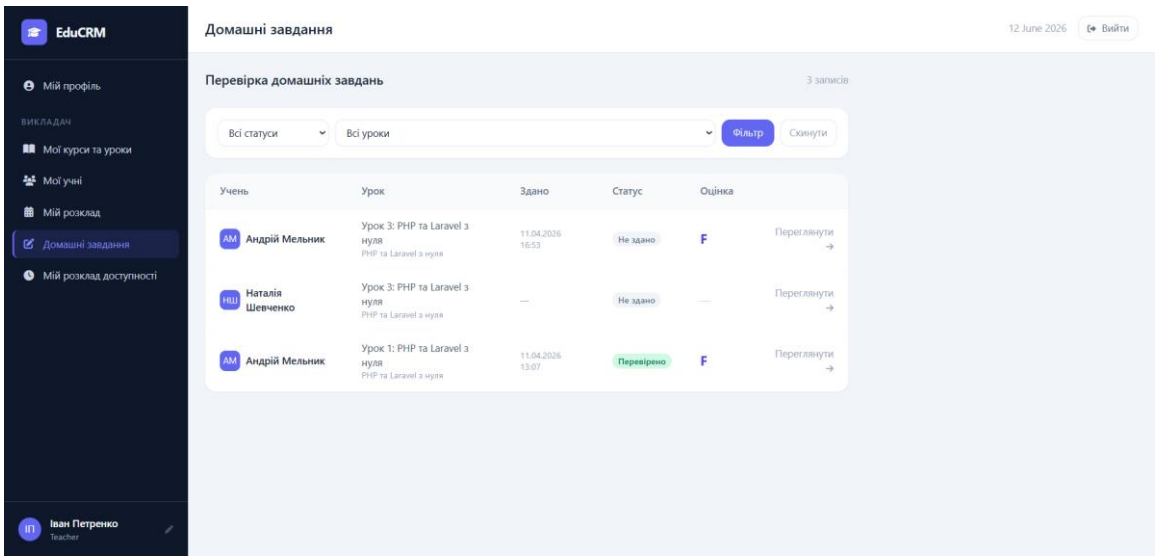


Рис. 3.10. Інтерфейс кабінету «Викладача»: модуль перевірки домашніх завдань

Кабінет студента є основним інструментом взаємодії клієнта зі школою. Він спроєктований максимально інтуїтивно і складається з трьох ключових розділів:

– мій розклад (student.schedule): головна сторінка студента. Вона відображає хронологічний список майбутніх занять (Scheduled) та вже проведених (Completed). За наявності згенерованого Zoom-посилання студент може приєднатися до уроку в один клік безпосередньо з системи;

– домашні завдання (student.homework): на цій сторінці студент бачить список призначених йому завдань. Для виконання він може завантажити файл умов, а після вирішення — завантажити свій архів або документ (через фасад Storage) та відправити його на перевірку. Тут також відображається фінальна оцінка та коментар від викладача після перевірки;
– оплата та баланс (student.payments): модуль фінансового білінгу. Студент (або його опікун/guardian) може переглянути поточний баланс занять і стан грошового рахунку. Інтерфейс пропонує вибрати один із доступних тарифних планів (tariff_plan_id) або ввести довільну суму для поповнення (initiateMoneyTopup). Після натискання кнопки оплати система перенаправляє клієнта на безпечну сторінку еквайрингу WayForPay. У разі успіху платежу, баланс уроків оновлюється автоматично.

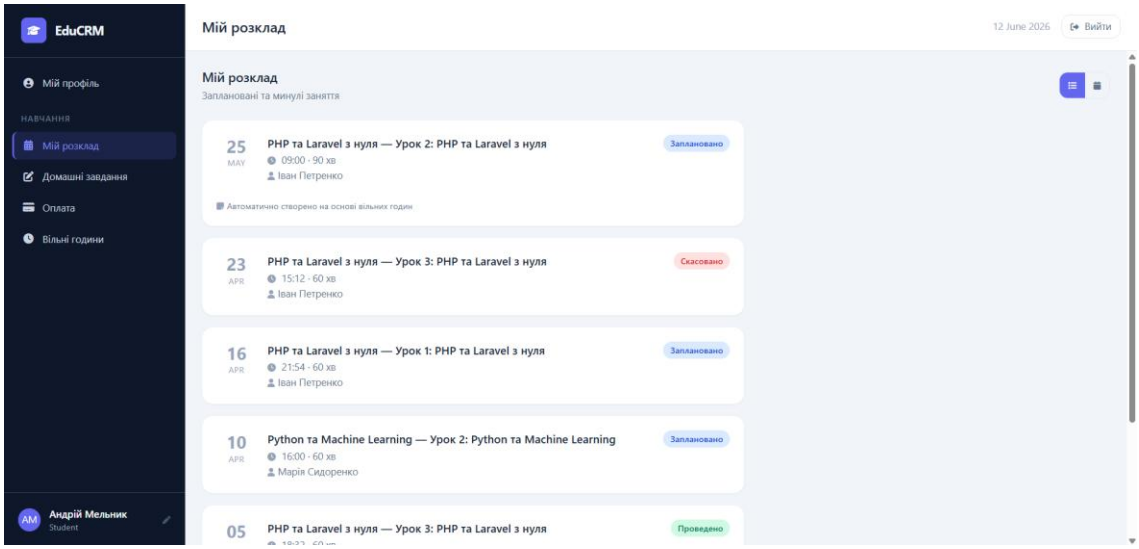


Рис. 3.11. Клієнтський портал «Студента»: перегляд розкладу

3.4 Функціональне тестування програмного продукту

Для забезпечення надійності, відмовостійкості та фінансової безпеки розробленої CRM/LMS-системи, фінальним етапом програмної реалізації стало автоматизоване тестування вихідного коду. Оскільки система оперує реальними грошовими транзакціями (інтеграція з WayForPay) та керує доступами різних типів користувачів, наявність тестів є запорукою уникнення регресійних помилок.

Основним інструментом для тестування було обрано фреймворк PHPUnit. Для гарантування ізольованості перевірок у кожному тестовому класі використано трейт RefreshDatabase, який автоматично очищає та перестворює базу даних у пам'яті після виконання кожного методу [29]. Згідно з концепцією "Піраміди тестування", розроблені тести поділяються на модульні (Unit) та функціональні (Feature) [30].

Модульні тести (розміщені у директорії tests/Unit/) призначені для перевірки ізольованої логіки сервісних класів без імітації HTTP-запитів. Основний акцент було зроблено на перевірці фінансових розрахунків та криптографії.

Для перевірки логіки нарахування балансів було створено клас LessonBalanceServiceTest. Цей набір тестів відповідає за те, щоб студенти отримували правильну кількість уроків або грошей після оплати:

- метод test_credit_increments_lessons_balance перевіряє базове додавання уроків до активного запису на курс (Enrollment);
- метод test_credit_from_payment_lessons_type_credits_lessons_balance перевіряє складний бізнес-процес: якщо платіж має тип lessons, система повинна знайти пов'язаний тарифний план (TariffPlan) і нарахувати саме ту кількість занять, яка в ньому вказана;
- метод test_credit_from_payment_uses_lessons_count_over_tariff перевіряє крайній випадок (Edge Case): якщо у самому платежі вказана ручна кількість уроків, вона повинна мати вищий пріоритет, ніж значення з тарифу.

Для перевірки безпеки платежів було розроблено WayForPayServiceTest, який тестує генерацію та перевірку HMAC_MD5 підписів. Лістинг нижче демонструє реалізацію тесту генерації валідного підпису для платіжної форми:

```
class WayForPayServiceTest extends TestCase
{
    use RefreshDatabase;

    private WayForPayService $service;
```

```
    private function configureSetting(): void
    {
        Setting::set('wfp_merchant_account', 'test_merchant');
        Setting::set('wfp_merchant_secret',  'test_secret');
        Setting::set('wfp_merchant_domain',  'example.com');
    }

    protected function setUp(): void
    {
        parent::setUp();
        $this->service = new WayForPayService();
    }
public function test_build_form_signature_is_valid_hmac(): void
{
    $this->configureSetting();

    $form = $this->service->buildForm([
        'order_reference' => 'REF-42',
        'order_date'      => 1700000000,
        'amount'          => 300,
        'product_name'    => 'Lesson',
        'product_count'   => 1,
        'product_price'   => 300,
    ]);

    $signString = implode(';', [
        'test_merchant', 'example.com', 'REF-42', 1700000000, 300,
'UAH', 'Lesson', 1, 300,
    ]);
    $expected = hash_hmac('md5', $signString, 'test_secret');

    $this->assertEquals($expected, $form['merchantSignature']);
}
}
```

Інші методи класу (test_verify_callback_returns_false_on_wrong_signature) гарантують, що зловмисник не зможе підробити Webhook-відповідь від банку, оскільки сервіс відхилить невалідний підпис.

Функціональні тести (tests/Feature/) імітують повний життєвий цикл запиту: проходження через маршрутизатор, перевірку прав доступу в Middleware, валідацію в контролері та запис у БД.

Для перевірки рольової моделі розроблено DashboardControllerTest. Він імітує авторизацію різних типів користувачів (за допомогою фабрик User::factory()->admin() тощо) і гарантує, що:

– «Адміністратори» та «Менеджери» залишаються на головному Дашборді (assertViewIs('dashboard.index'));

– «Викладачі» автоматично перенаправляються на сторінку своїх студентів (assertRedirect(route('teacher.students')));
– «Студенти» перенаправляються на свій розклад.

Найважливішим функціональним тестом є WayForPayControllerTest, який перевіряє HTTP-взаємодію з платіжним шлюзом. Зокрема, метод test_callback_does_not_double_credit_already_paid_payment гарантує ідемпотентність системи — якщо банк надішле Webhook про успішну оплату двічі, система не нарахує гроші студенту повторно.

Лістинг тестування процесу ініціалізації платежу (test_initiate_creates_pending_payment_and_returns_redirect_view):

```
class WayForPayControllerTest extends TestCase
{
    use RefreshDatabase;
public function
test_initiate_creates_pending_payment_and_returns_redirect_view():
void
{
    $this->configureWfp();

    $student    = User::factory()->student()->create();
    $enrollment = Enrollment::factory()->create(['student_id' =>
$student->id]);
    $tariff     = TariffPlan::factory()->create(['status' =>
'active', 'count_lessons' => 4, 'amount' => 800]);

    $response = $this->actingAs($student)
        ->post(route('wayforpay.initiate'), [
            'enrollment_id'  => $enrollment->id,
            'tariff_plan_id' => $tariff->id,
        ]);

    $response->assertOk()->assertViewIs('wayforpay.redirect');

    $this->assertDatabaseHas('payments', [
        'student_id'    => $student->id,
        'enrollment_id' => $enrollment->id,
        'status'        => 'pending',
        'method'        => 'online',
        'amount'        => 800,
    ]);
}
```

У таблиці 3.3 зведено структуру та зону відповідальності розроблених тестових наборів.

Таблиця 3.3

Опис розроблених наборів автоматизованих тестів та їх призначення

Назва тестового класу	**Тип тесту**	**Зона відповідальності**
`LessonBalanceServiceTest`	Unit	Логіка додавання/списання уроків та грошей (`money_balance`) студента.
`WayForPayServiceTest`	Unit	Валідація алгоритму генерації хешу HMAC_MD5 для безпеки транзакцій.
`UserModelTest`, `ScheduleModelTest`	Unit	Коректність зв'язків ORM Eloquent, статусів (`draft`) та хелперів доступу.
`DashboardControllerTest`	Feature	Маршрутизація та перенаправлення (Redirects) залежно від ролі користувача.
`GenerateDraftSchedulesCommandTest`	Feature	Робота консольної команди, алгоритм зіставлення вільного часу викладача та груп.
`StudentHomeworkTest`	Feature	Валідація форм, завантаження файлів-розв'язків студентом у файлову систему.
`WayForPayControllerTest`	Feature	Ідемпотентність вебхуків банку; запобігання подвійному зарахуванню коштів.
`TeacherAvailabilityTest`, `StudentAvailabilityTest`	Feature	Збереження матриці вільного часу викладачів та студентів у БД.
`StudentPaymentsTest`, `StudentScheduleTest`	Feature	Рендеринг Blade-шаблонів (HTTP 200) для перегляду розкладу та фінансів студента.

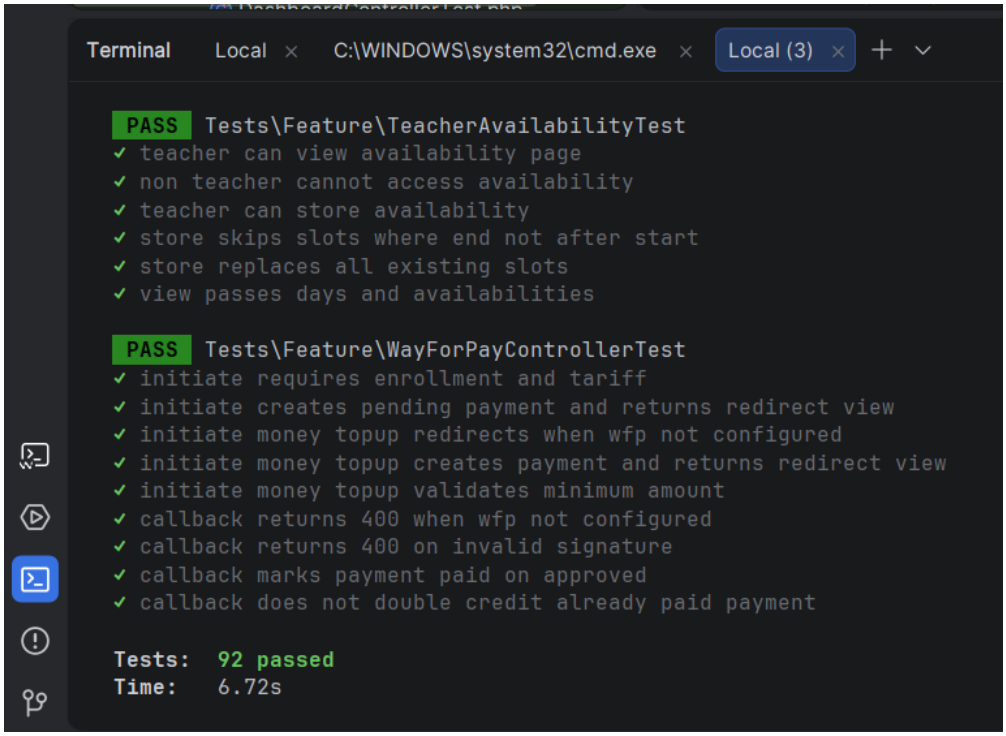


Рис. 3.12. Результат успішного виконання набору автоматизованих тестів

Для систематизації процесу тестування та підготовки його до інтеграції у CI/CD-пайплайни, було побудовано UML-діаграму діяльності. Вона наочно демонструє процес запуску тестів перед злиттям нового коду з основною гілкою репозиторію.

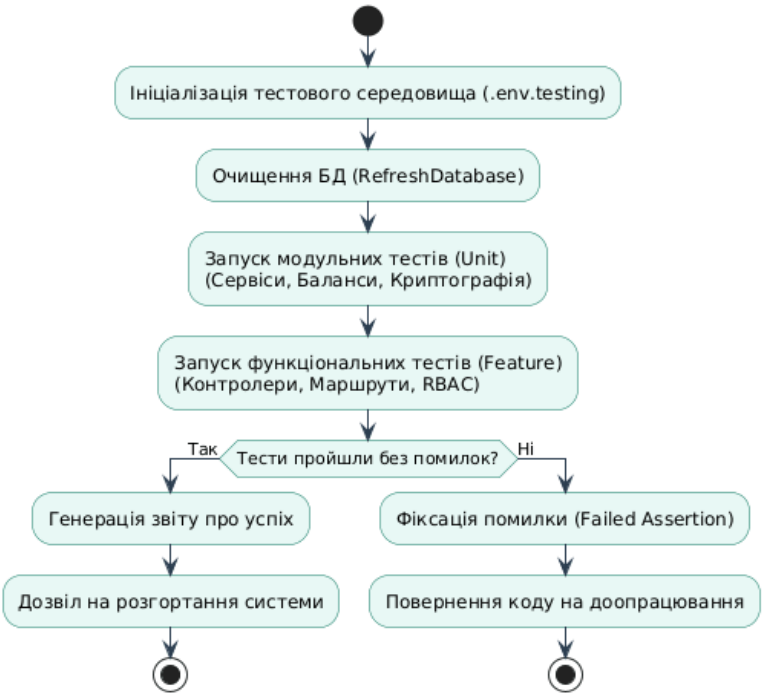


Рис. 3.13. UML-діаграма активності процесу автоматизованого тестування програмного забезпечення

Проведене тестування повністю підтвердило стабільність бізнес-логіки. Воно доводить, що система здатна безпечно обробляти фінансові дані, стійка до логічних помилок і має надійний захист від спроб несанкціонованого доступу.

Висновки до третього розділу

У третьому розділі було виконано практичну реалізацію, розгортання інтерфейсу та комплексне тестування розробленої гібридної CRM/LMS-системи для автоматизації процесів IT-школи. Отримані результати підтвердили працездатність, безпечність та відповідність системи поставленим вимогам.

Серверну частину системи реалізовано на основі Laravel та мови програмування PHP 8 із дотриманням стандартів PSR-12. Було впроваджено архітектурний патерн Service-Repository, що дозволив розділити бізнес-логіку, роботу з даними та контролери, підвищивши гнучкість, масштабованість і зручність підтримки коду. Використання Dependency Injection та сучасних можливостей PHP дало змогу оптимізувати структуру застосунку.

Особливу увагу приділено безпеці та цілісності даних. Реалізовано багаторівневий захист, який включає валідацію вхідних даних, рольову модель доступу (RBAC) та двофакторну автентифікацію (2FA). Для захисту фінансових операцій через WayForPay використано транзакції та механізми блокування даних, що дозволило уникнути помилок під час одночасної обробки платежів. Клієнтська частина була розроблена за допомогою Blade, Tailwind CSS та серверного рендерингу, що забезпечило швидку роботу та зручний адаптивний інтерфейс.

Для перевірки стабільності системи було розроблено 12 автоматизованих тестів у PHPUnit. Проведене тестування підтвердило коректність роботи ключових модулів: фінансових розрахунків, обробки платежів, генерації розкладів, LMS-функціоналу та захисту від повторної обробки вебхуків.

Таким чином розроблена система повністю відповідає функціональним та архітектурним вимогам, забезпечує високу продуктивність, безпеку даних та стабільність роботи в реальних умовах експлуатації.

ВИСНОВКИ

У кваліфікаційній роботі вирішено актуальну інженерно-практичну задачу — спроєктовано, розроблено та протестовано гібридну CRM/LMS-систему для автоматизації академічних, адміністративних і фінансових процесів сучасної IT-школи. Отримані результати базуються на поєднанні теоретичного аналізу предметної області, архітектурного проєктування та повноцінної програмної реалізації.

У процесі дослідження предметної області виявлено ключові проблеми існуючих підходів до управління освітніми центрами, зокрема використання розрізнених інструментів, що призводить до помилок у розкладах, втрати контролю над фінансами та ускладнення комунікації між учасниками навчального процесу. Обґрунтовано необхідність розробки спеціалізованого програмного рішення та здійснено вибір технологічного стеку, що включає PHP, Laravel, Tailwind CSS та MySQL, які забезпечують надійність, масштабованість і гнучкість системи.

Архітектура системи побудована на основі принципів об’єктно-орієнтованого проєктування та стандартів SOLID із застосуванням патерну Service-Repository. Такий підхід дозволив чітко розмежувати рівні відповідальності між контролерами, сервісами та шаром доступу до даних. Спроєктовано реляційну базу даних із понад 10 взаємопов’язаними сутностями, нормалізовану та оптимізовану для забезпечення цілісності даних. Додатково розроблено UML-діаграми, які формалізують ключові процеси системи, включаючи управління розкладами, навчальними групами та фінансовими операціями.

Практична реалізація системи включає інтеграцію із зовнішніми сервісами, такими як Zoom для організації онлайн-занять та платіжний сервіс WayForPay для автоматизації фінансових операцій. Реалізовано автоматизовані процеси через консольні команди, які дозволяють формувати розклади, керувати навчальними процесами та обробляти системні задачі у фоновому режимі.

Особливу увагу приділено забезпеченню цілісності фінансових даних та захисту від помилок при одночасному доступі користувачів.

Система безпеки реалізована на декількох рівнях і включає рольову модель доступу (RBAC), механізм двофакторної автентифікації (2FA), а також захист критичних операцій за допомогою транзакцій бази даних і блокувань від стану гонитви. Це дозволяє гарантувати коректність фінансових операцій та стабільність роботи системи навіть за умов високого навантаження.

Користувацький інтерфейс реалізовано з використанням серверного рендерингу, шаблонізатора Blade та Tailwind CSS. Система містить чотири окремі ролі користувачів із різними рівнями доступу та функціональністю, що забезпечує зручність використання та логічну ізоляцію робочих середовищ. Для викладачів реалізовано додаткові інструменти контролю навчального процесу, а для студентів — доступ до розкладів, матеріалів і фінансової інформації.

Якість та стабільність системи підтверджено шляхом створення та виконання набору автоматизованих тестів у PHPUnit. Тестове покриття охоплює ключові модулі системи, включаючи фінансові розрахунки, LMS-функціонал, обробку вебхуків та консольні команди. Використання ізольованого середовища тестування дозволило забезпечити надійність результатів та відсутність впливу на основні дані.

У підсумку розроблена CRM/LMS-система демонструє високу продуктивність, безпеку та готовність до впровадження в реальні умови експлуатації. Вона дозволяє суттєво автоматизувати процеси управління IT-школою, зменшити вплив людського фактора, підвищити ефективність фінансового обліку та покращити якість організації навчального процесу.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Bates A. W. Teaching in a Digital Age: Guidelines for Designing Teaching and Learning. – 2nd ed. – Vancouver: Tony Bates Associates Ltd, 2019.
2. Siemens G., Gašević D., Dawson S. Preparing for the digital university: a review of current state of online learning. – Athabasca University Press, 2015.
3. Kostolányová K., Simonová I. Learning Management Systems and their potential for personalizing education. // International Journal of Information and Communication Technologies in Education. – 2020.
4. Buttle F., Maklan S. Customer Relationship Management: Concepts and Technologies. – 4th ed. – Routledge, 2019.
5. Nguyen T. The Effectiveness of Online Learning: Beyond No Significant Difference and Future Horizons. // MERLOT Journal of Online Learning and Teaching. – 2015.
6. Dahlstrom, E., Brooks, D. C., & Bichsel, J. (2014). The Current Ecosystem of Learning Management Systems in Higher Education. EDUCAUSE Research Report.
7. Turnbull, W. P., Chugh, R., & Luck, J. (2019). Learning Management Systems: An Overview. Encyclopedia of Education and Information Technologies, 1052-1058.
8. Weaver, D., Spratt, C., & Farley, L. S. (2008). Keeping track: the Moodle gradebook. Proceedings of the 25th ASCILITE Conference, 1074-1078.
9. Greenberg, P. (2010). CRM at the Speed of Light: Social CRM Strategies, Tools, and Techniques for Engaging Your Customers. McGraw-Hill Osborne Media.
10. Heggart, K. R., & Yoo, J. (2018). Getting the most from Google Classroom: A pedagogical framework for tertiary educators. Australian Journal of Teacher Education, 43(3), 140-153.
11. Sommerville, I. (2015). Software Engineering. 10th ed. Pearson.
12. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.

13. Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws. 2nd ed. Wiley.
14. Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly Media.
15. Sommerville, I. (2015). Software Engineering. 10th ed. Pearson.
16. Otwell, T. (2023). Laravel 10.x Documentation. Laravel LLC.
17. Mutchler, M. (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). Internet Engineering Task Force (IETF).
18. Wathan, A. (2023). Tailwind CSS Documentation. Tailwind Labs.
19. Bergmann, S. (2023). PHPUnit Manual.
20. PHP Framework Interop Group (PHP-FIG). (2019). PSR-12: Extended Coding Style Guide. [Електронний ресурс].
21. Otwell, T. (2023). Laravel 10.x Documentation: Validation. Laravel LLC. [Електронний ресурс].
22. RFC 6238. TOTP: Time-Based One-Time Password Algorithm. Internet Engineering Task Force (IETF), 2011. [Електронний ресурс].
23. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
24. MySQL Documentation. InnoDB Locking and Transaction Model. Oracle Corporation. [Електронний ресурс].
25. Zoom Video Communications, Inc. (2023). Server-to-Server OAuth API Documentation. [Електронний ресурс].
26. Stauffer, M. (2019). Laravel Up & Running: A Framework for Building Modern PHP Apps. O'Reilly Media. [Електронний ресурс].
27. Tailwind Labs Inc. (2023). Tailwind CSS: Play CDN Documentation. [Електронний ресурс].
28. Marcotte, E. (2011). Responsive Web Design. A Book Apart. [Електронний ресурс].
29. Bergmann, S. (2023). PHPUnit Manual: The PHP Testing Framework. [Електронний ресурс].
30. Fowler, M. (2012). The Practical Test Pyramid. [Електронний ресурс].

ДОДАТКИ

Додаток А

Загальні відомості

1. Повна назва системи: гібридна CRM/LMS-система для автоматизації бізнес-процесів та академічного обліку IT-школи. Призначення: автоматизація розкладів, фінансового білінгу, управління навчальними групами та перевірки домашніх завдань.

2. Вимоги до функціональних характеристик Система повинна забезпечувати роботу чотирьох типів користувачів із розмежуванням прав доступу (RBAC):

— «Адміністратор»: повний доступ до всіх модулів, налаштування глобальних параметрів (API ключі WayForPay, Zoom), управління тарифами, перегляд фінансової аналітики;
— «Менеджер»: управління студентами (Enrollments), формування навчальних груп, контроль платежів та балансів;
— «Викладач»: управління особистим розкладом (встановлення вільних годин), перегляд списку своїх груп/студентів, отримання та перевірка домашніх завдань;
— «Студент»: перегляд особистого розкладу із посиланнями на Zoom-конференції, завантаження розв'язків домашніх завдань, поповнення балансу через платіжний шлюз.

3. Вимоги до програмного забезпечення:

— серверна частина (Back-end): PHP 8+, фреймворк Laravel 10.x;
— клієнтська частина (Front-end): шаблонізатор Blade, Tailwind CSS, нативний JavaScript;
— база даних: MySQL 8.0+ (рушій InnoDB).

Структура реляційної бази даних та лістинги міграцій

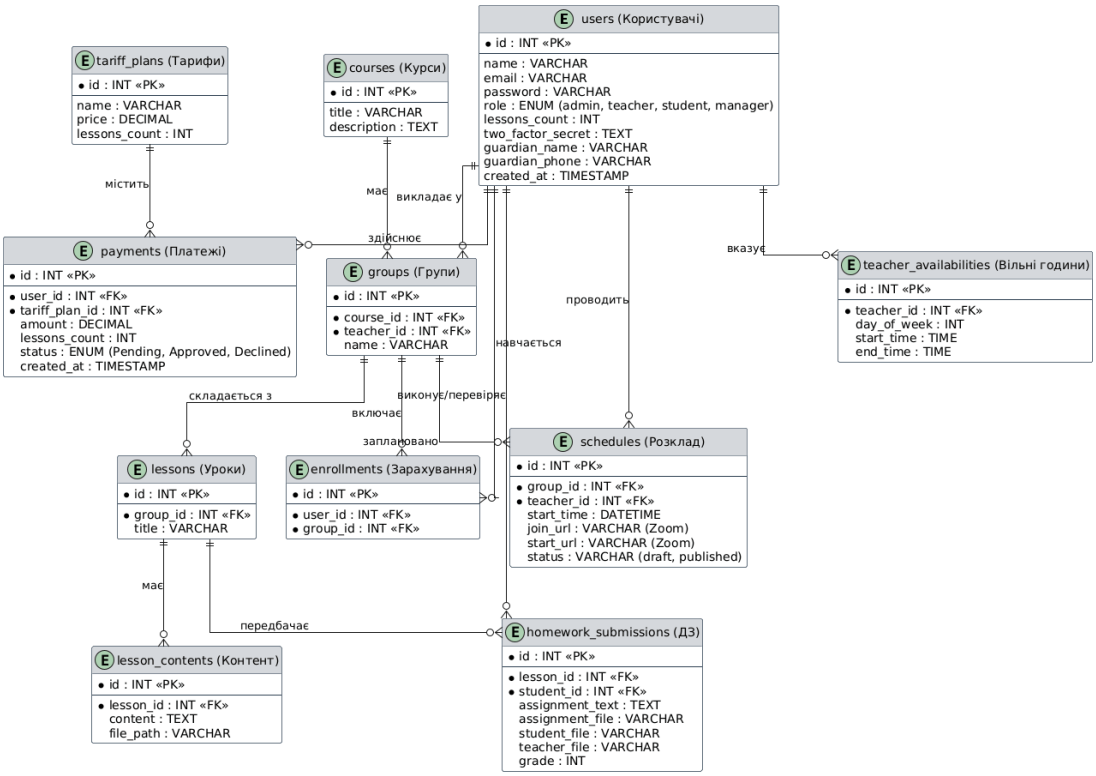


Рис.1 ER-діаграма бази даних

Міграція створення таблиці запису студентів на курси (enrollments):

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('enrollments', function (Blueprint $table)
{
            $table->id();
            $table->unsignedBigInteger('student_id');
            $table->unsignedBigInteger('group_id');
            $table->string('status')->default('active');
            $table->date('enrolled_at');
            $table->text('notes')->nullable();
            $table->foreign('student_id')->references('id')-
>on('users')->cascadeOnDelete();
            $table->foreign('group_id')->references('id')-
>on('groups')->cascadeOnDelete();
            $table->unique(['student_id', 'group_id']);
```

```
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('enrollments');
    }
};
```

Міграція створення таблиці розкладів занять (schedules):

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('schedules', function (Blueprint $table) {
            $table->id();
            $table->foreignId('lesson_id')->constrained()-
>cascadeOnDelete();
            $table->foreignId('student_id')->constrained('users')-
>cascadeOnDelete();
            $table->foreignId('teacher_id')->constrained('users')-
>cascadeOnDelete();
            $table->date('scheduled_date');
            $table->time('scheduled_time');
            $table->unsignedSmallInteger('duration_minutes')-
>default(60);
            $table->enum('status',
['scheduled','completed','cancelled','no_show'])-
>default('scheduled');
            $table->string('zoom_link')->nullable();
            $table->text('notes')->nullable();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
```

```
    {
        Schema::dropIfExists('schedules');
    }
};
```

Міграція створення таблиці здачі домашніх завдань (homework_submissions):

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('homework_submissions', function (Blueprint
$table) {
            $table->id();
            $table->unsignedBigInteger('lesson_id');
            $table->unsignedBigInteger('student_id');
            $table->unsignedBigInteger('schedule_id')->nullable();
            $table->text('content')->nullable();
            $table->string('file_path')->nullable();
            $table->string('status')->default('pending'); //
pending, submitted, reviewed
            $table->string('grade')->nullable();
            $table->text('teacher_comment')->nullable();
            $table->timestamp('submitted_at')->nullable();
            $table->timestamp('reviewed_at')->nullable();
            $table->foreign('lesson_id')->references('id')-
>on('lessons')->cascadeOnDelete();
            $table->foreign('student_id')->references('id')-
>on('users')->cascadeOnDelete();
            $table->foreign('schedule_id')->references('id')-
>on('schedules')->nullOnDelete();
            $table->unique(['lesson_id', 'student_id']);
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('homework_submissions');
}
};
```

Додаток В

Лістинги програмного коду

Програмна реалізація контролера платежів та обробки Webhook (WayForPayController):

```
<?php

namespace App\Http\Controllers;

use App\Models\Enrollment;
use App\Models\Payment;
use App\Models\TariffPlan;
use App\Services\LessonBalanceService;
use App\Services\WayForPayService;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

class WayForPayController extends Controller
{
    public function __construct(
        private WayForPayService $wfp,
        private LessonBalanceService $balance
    ) {}

    public function initiate(Request $request)
    {
        $request->validate([
            'enrollment_id'  => 'required|exists:enrollments,id',
            'tariff_plan_id' => 'required|exists:tariff_plans,id',
        ]);

        $enrollment = Enrollment::where('id', $request-
>enrollment_id)
            ->where('student_id', auth()->id())
            ->firstOrFail();

        $tariff      = TariffPlan::where('id', $request-
>tariff_plan_id)->where('status', 'active')->firstOrFail();
        $orderRef    = 'ORDER-' . auth()->id() . '-' . time();
        $productName = $tariff->name;

        $payment = Payment::create([
            'student_id'     => auth()->id(),
            'enrollment_id'  => $enrollment->id,
            'tariff_plan_id' => $tariff->id,
            'lessons_count'  => $tariff->count_lessons,
            'amount'         => $tariff->amount,
            'status'         => 'pending',
            'method'         => 'online',
            'payment_date'   => now()->toDateString(),
            'notes'          => "WayForPay: {$productName}
```

```
({$tariff->count_lessons} уроків) | ref:{$orderRef}",
        ]);

        $form = $this->wfp->buildForm([
            'order_reference' => $orderRef,
            'order_date'      => $payment->created_at->timestamp,
            'amount'          => $tariff->amount,
            'product_name'    => $productName,
            'product_count'   => 1,
            'product_price'   => $tariff->amount,
        ]);

        return view('wayforpay.redirect', compact('form'));
    }

    public function initiateMoneyTopup(Request $request)
    {
        $request->validate([
            'amount' => 'required|numeric|min:10',
        ]);

        if (!$this->wfp->isConfigured()) {
            return redirect()->route('student.payments')
                ->with('error', 'Онлайн-оплата не налаштована.
Оберіть інший метод.');
        }

        $amount   = (float) $request->amount;
        $orderRef = 'MONEY-' . auth()->id() . '-' . time();

        $payment = Payment::create([
            'student_id'   => auth()->id(),
            'payment_type' => 'money',
            'amount'       => $amount,
            'status'       => 'pending',
            'method'       => 'online',
            'payment_date' => now()->toDateString(),
            'notes'        => "WayForPay: поповнення балансу |
ref:{$orderRef}",
        ]);

        $form = $this->wfp->buildForm([
            'order_reference' => $orderRef,
            'order_date'      => $payment->created_at->timestamp,
            'amount'          => $amount,
            'product_name'    => 'Поповнення грошового балансу',
            'product_count'   => 1,
            'product_price'   => $amount,
        ]);

        return view('wayforpay.redirect', compact('form'));
    }
```

```
    public function callback(Request $request)
    {
        $data = $request->all();
        Log::info('WayForPay callback', $data);

        if (!$this->wfp->isConfigured()) {
            return response()->json(['error' => 'not configured'],
400);
        }

        if (!$this->wfp->verifyCallback($data)) {
            Log::warning('WayForPay invalid signature', $data);
            return response()->json(['error' => 'invalid
signature'], 400);
        }

        $orderRef = $data['orderReference'] ?? null;
        $status   = $data['transactionStatus'] ?? null;

        $payment = Payment::where('notes', 'like',
"%ref:{$orderRef}%")->first();

        if ($payment && $status === 'Approved' && $payment->status
!== 'paid') {
            $payment->update(['status' => 'paid']);
            $this->balance->creditFromPayment($payment-
>fresh(['tariffPlan', 'enrollment']));
        }

        $time = now()->timestamp;
        return response()->json(
            $this->wfp->buildCallbackResponse($orderRef, 'accept',
(string) $time)
        );
    }
}
```

Алгоритм консольної команди для автоматичної генерації розкладів (GenerateDraftSchedules):

```
<?php

namespace App\Console\Commands;

use App\Models\Enrollment;
use App\Models\Lesson;
use App\Models\Schedule;
use App\Models\TeacherAvailability;
use Carbon\Carbon;
use Illuminate\Console\Command;
use Illuminate\Support\Collection;

class GenerateDraftSchedules extends Command
{
```

```
    protected $signature   = 'schedules:generate-drafts {--days=14
: How many days ahead to generate}';
    protected $description = 'Auto-generate draft schedules based
on teacher and student availability overlaps';

    public function handle(): int
    {
        $days     = (int) $this->option('days');
        $today    = Carbon::today();
        $horizon  = $today->copy()->addDays($days);
        $created  = 0;
        $skipped  = 0;

        $enrollments = Enrollment::with(['group.course',
'group.teacher'])
            ->where('status', 'active')
            ->get();

        foreach ($enrollments as $enrollment) {
            $group = $enrollment->group;
            if (!$group || !$group->teacher_id || !$group-
>course_id) {
                continue;
            }

            $teacherId = $group->teacher_id;
            $studentId = $enrollment->student_id;
            $courseId  = $group->course_id;

            // Load availabilities indexed by day_of_week
            $teacherAvail =
TeacherAvailability::where('teacher_id', $teacherId)
                ->where('is_active', true)
                ->get()
                ->groupBy('day_of_week');

            $studentAvail =
TeacherAvailability::where('teacher_id', $studentId)
                ->where('is_active', true)
                ->get()
                ->groupBy('day_of_week');

            // Skip if either side has no availability set
            if ($teacherAvail->isEmpty() || $studentAvail-
>isEmpty()) {
                continue;
            }

            // Iterate over each future date in the horizon
            for ($date = $today->copy()->addDay(); $date-
>lte($horizon); $date->addDay()) {
                $dow = $date->isoWeekday(); // 1=Mon … 7=Sun
```

```
                $tSlots = $teacherAvail->get($dow, collect());
                $sSlots = $studentAvail->get($dow, collect());

                if ($tSlots->isEmpty() || $sSlots->isEmpty()) {
                    continue;
                }

                $overlaps = $this->findOverlaps($tSlots, $sSlots);

                foreach ($overlaps as $overlapStart) {
                    // Avoid duplicate schedule for same
student/date/time
                    $exists = Schedule::where('student_id',
$studentId)
                        ->where('scheduled_date', $date-
>toDateString())
                        ->where('scheduled_time', $overlapStart)
                        ->exists();

                    if ($exists) {
                        $skipped++;
                        continue;
                    }

                    // Find next lesson for this student in this
course
                    $usedLessonIds = Schedule::where('student_id',
$studentId)
                        ->whereHas('lesson', fn($q) => $q-
>where('course_id', $courseId))
                        ->whereIn('status', ['scheduled',
'completed', 'draft'])
                        ->pluck('lesson_id');

                    $nextLesson = Lesson::where('course_id',
$courseId)
                        ->whereNotIn('id', $usedLessonIds)
                        ->orderBy('order')
                        ->first();

                    if (!$nextLesson) {
                        break; // No more lessons in course for
this student
                    }

                    Schedule::create([
                        'lesson_id'        => $nextLesson->id,
                        'student_id'       => $studentId,
                        'teacher_id'       => $teacherId,
                        'scheduled_date'   => $date-
>toDateString(),
                        'scheduled_time'   => $overlapStart,
                        'duration_minutes' => $nextLesson-
```

```
>duration_minutes ?? 60,
                        'status'           => 'draft',
                        'notes'            => 'Автоматично
створено на основі вільних годин',
                    ]);

                    $created++;
                }
            }
        }

        $this->info("Done. Created: {$created}, Skipped (already
exist): {$skipped}");
        return self::SUCCESS;
    }

    /**
     * Find the start times of all overlapping intervals between
teacher and student slots.
     * Returns an array of time strings "HH:MM" representing
overlap start times.
     */
    private function findOverlaps(Collection $teacherSlots,
Collection $studentSlots): array
    {
        $overlaps = [];

        foreach ($teacherSlots as $t) {
            foreach ($studentSlots as $s) {
                $start = max($t->start_time, $s->start_time);
                $end   = min($t->end_time,   $s->end_time);

                if ($start < $end) {
                    // Use the start of overlap as the lesson time
                    $overlaps[] = substr($start, 0, 5); // "HH:MM"
                }
            }
        }

        return array_unique($overlaps);
    }
}
```

Додаток Г

Керівництво системного адміністратора (Інструкція з розгортання)

Вимоги до сервера. Для коректної роботи системи сервер повинен підтримувати такі технології:

- PHP >= 8.1;
- Composer >= 2.0;
- MySQL >= 8.0;
- Node.js >= 16.x та NPM.

Покрокова інструкція з розгортання платформи (Deployment)

Клонування репозиторію:

завантажити вихідний код системи з системи контролю версій Git:

git clone https://github.com/pashkotal/crm-school.git

cd crm-school

Встановлення залежностей:

встановити PHP-бібліотеки та фронтенд-залежності:

composer install --optimize-autoloader --no-dev

npm install

npm run build

Конфігурація середовища:

створити файл конфігурації з базового шаблону та згенерувати криптографічний ключ додатку:

cp .env.example .env

php artisan key:generate

У файлі .env необхідно вказати актуальні доступи до бази даних (DB_DATABASE, DB_USERNAME, DB_PASSWORD) та облікові дані платіжного шлюзу (WFP_MERCHANT_ACCOUNT, WFP_MERCHANT_SECRET).

Ініціалізація бази даних:

запустити міграції для створення структури бази даних та наповнити її початковими даними (ролі, обліковий запис адміністратора):

php artisan migrate –seed

Налаштування планувальника (Cron):

для автоматичної генерації розкладів необхідно додати команду виконання завдань Laravel у системний планувальник (crontab):

* * * * * cd /path-to-your-project && php artisan schedule:run >> /dev/null 2>&1

Налаштування прав доступу:

надати вебсерверу права на запис у директорії кешу та завантажень:

chmod -R 775 storage bootstrap/cache

php artisan storage:link